

Honeywell

TotalPlant Solution (TPS) System

HCI Client Developer's Guide

TP60-400

R400

03/2007

Notices and Trademarks

**Copyright 1998 - 2007 by Honeywell International Inc.
Release 400 March 22, 2007**

While this information is presented in good faith and believed to be accurate, Honeywell disclaims the implied warranties of merchantability and fitness for a particular purpose and makes no express warranties except as may be stated in its written agreement with and for its customers.

In no event is Honeywell liable to anyone for any indirect, special or consequential damages. The information and specifications in this document are subject to change without notice.

Honeywell and **TotalPlant** are registered trademarks of Honeywell International Inc.

Other brand or product names are trademarks of their respective owners.

Honeywell International
Process Solutions
2500 West Union Hills Drive
Phoenix, AZ 85027

About This Document

This document describes tools, techniques, and procedures of interest to those users tasked with the development of Honeywell Communications Interface (HCI) client applications. It describes the facilities available to the client application developer and the Application Program Interface (API) for client applications.

It is assumed that the reader is familiar with C++ or Visual Basic, and is engaged in the development of HCI client application software

Release Information

Document Name	Document ID	Release Number	Publication Date
HCI Client Developer's Guide - TP60	TP60-400	400	03/2007

References

The following list identifies all documents that may be sources of reference for material discussed in this publication.

Document Title	Doc ID
<i>System Management Operations Guide</i>	EX07
<i>System Management Configuration Guide</i>	EX08
<i>TPS System Administration Guide</i>	TP06X
<i>TPS System Implementation Guide</i>	TP08X
<i>TPN Server User's Guide</i>	TP11X
<i>TPS System Configuration Utility User's Guide</i>	TP20X
<i>OPC Specification Reference Manual</i>	TP41

Contacts

World Wide Web

The following lists Honeywell's World Wide Web sites that will be of interest to our Industry Solutions customers.

Honeywell Organization	WWW Address (URL)
Corporate	http://www.honeywell.com
Process Solutions	http://www.honeywell.com/ps

Telephone

Contact us by telephone at the numbers listed below.

Organization		Phone Number	
United States and Canada	Honeywell International Inc. Industry Solutions	1-800-343-0228	Sales
		1-800-525-7439	Service
		1-800-822-7673	Technical Support
Asia Pacific	Honeywell Asia Pacific Inc. Hong Kong	(852) 23 31 9133	
Europe	Honeywell PACE Brussels, Belgium	[32-2] 728-2711	
Latin America	Honeywell International Inc. Sunrise, Florida U.S.A.	(954) 845-2600	

Symbol Definitions

The following table lists those symbols used in this document to denote certain conditions.

Symbol	Definition
	ATTENTION: Identifies information that requires special consideration.
	TIP: Identifies advice or hints for the user, often in terms of performing a task.
	REFERENCE -EXTERNAL: Identifies an additional source of information outside of the bookset.
	REFERENCE - INTERNAL: Identifies an additional source of information within the bookset.
CAUTION	Indicates a situation which, if not avoided, may result in equipment or work (data) on the system being damaged or lost, or may result in the inability to properly operate the process.
	CAUTION: Indicates a potentially hazardous situation, which if not avoided, may result in minor or moderate injury. It may also be used to alert against unsafe practices.
	WARNING: Indicates a potentially hazardous situation, which if not avoided, could result in serious injury or death.

Contents

1.	Getting Started.....	15
1.1	Introduction	15
	Purpose.....	15
	Abstract.....	15
	Features.....	16
	OPC Data Access 2.05 custom interfaces	16
	OPC Data Access 1.0A automation interfaces.....	17
	OPC Data Access 2.02 automation objects	17
	OPC Security 1.0 custom interface	17
1.2	Requirements.....	18
	Target audience and prerequisite skill recommendations	18
	Operating system	18
	Hardware environment.....	18
	Software environment	19
	Network environment	19
	External references	19
	About this guide	20
2.	Installation Verification	21
2.1	Verifying the HCI Client Developer Toolkit Installation	21
	Overview	21
	Verification of installed directories.....	22
	Subdirectory summary	23
	Procedure to configure the HCI sample server	31
	Other system updates	37
2.2	Developing HCI Clients on Non-TPS Nodes	38
	Overview	38
	Procedures to configure a non-TPS node for HCI client development	39
2.3	Accessing HCI Server From Remote Client	43
	Client considerations.....	43
	Procedure to set up client application access to remote HCI servers	43
2.4	Custom Client Installation.....	46
	Overview	46
	Introduction	46
	TPS installation requirements	47
	Security	47
	Startup	47
	Installation package guidelines	49

Contents

2.5 Verifying Installation and Configuration on a TPS Node	50
Monitor TPS Domains	50
Start/Stop the HCI Components	50
3. Key Concepts	51
3.1 Honeywell Communications Interface (HCI)	51
Communications Infrastructure overview	51
3.2 HCI Client Access	51
Overview	51
Create Instance	51
Conformance with OPC specification	53
Asynchronous	53
COM Client security considerations	54
Method Access security	61
Server Attributes	61
3.3 HCI Utilities	63
Overview	63
Asynchronous Utility classes	63
Data Manipulation Utility classes	64
4. Developing an HCI Client Application	65
4.1 Overview	65
4.2 Naming Conventions and Standards	65
TPS naming structure	65
4.3 Threading Model	66
4.4 Using the Custom Interface vs. the Automation Interface	66
Which interface should the client application use	66
4.5 Custom Application Scenario	66
Using Custom Interface	66
4.6 Automation Application Scenario (OPC 1.0A)	68
Using Automation Interface	68
4.7 Automation Application Scenario (OPC 2.0)	69
Using Automation Interface	69
5. HciTimeout Object	71
5.1 Overview	71
Introduction	71
5.2 HciTimeout	71
Functional description	71

Public Methods summary	73
AnyTimedCustomMethod	77
CheckStatus syntax and description	84
AnyTimedAutomationMethod	87
Status	91
6. Custom Interface APIs	99
6.1 Overview	99
Introduction	99
6.2 HCI Class	99
Functional description	99
Public Methods list	99
GetComponentInfo	100
GetCurrentDomainName	101
GetLocalHostName	102
IsLocalHost	103
6.3 IHciClientUtil	104
Functional description	104
Connecting to HciClientUtil COM object	104
Public Methods Summary	104
GetComponentInfo	105
GetCurrentDomainName	106
GetLocalHostName	107
IsLocalHost	108
HresultToString	109
6.4 IHciAttributes Interface	110
Functional Description	110
Public Methods Summary	110
SetAttributes	111
GetAttributes	113
6.5 IHciPulse Interface	115
Purpose	115
Functional description	115
Public Methods summary	116
Configure	116
Start	118
CancelPulse	119
GetInfo	120
6.6 HciSink Class	121
HciSink	123
~HciSink	123
Dadvise	124
Dunadvise	126
OnDataChange	127

Contents

GetDataObject.....	127
SetDataObject.....	128
QueryInterface.....	129
AddRef	129
Release	129
6.7 HciStreamUtil Class	130
Purpose.....	130
Public Methods summary	130
HciStreamUtil	131
~HciStreamUtil	132
GetByIndex (for OPCSTMFORMATDATETIME).....	133
GetByIndex (for OPCSTMFORMATDATA)	134
GetByIndex (for OPCSTMFORMATWRITECOMPLETE)	135
GetByHandle (for OPCSTRMFORMATDATETIME)	136
GetByHandle (for OPCSTRMFORMATDATA)	137
GetByHandle (for OPCSTRMFORMATWRITECOMPLETE)	138
GetGroupHeader.....	139
GetItemCount.....	140
GetStreamFormat.....	140
GetTransID.....	141
7. Automation Interface APIs.....	143
7.1 Introduction.....	143
Overview	143
HCI wrapper classes	143
7.2 ICHciVBHelper Interface	144
Functional description	144
Development notes	144
Public Methods summary	144
CreateHciObject.....	144
CreateRemoteObject.....	146
GetComponentInfo	147
UTCToLocal	148
CheckPoint.....	149
SetAttributes.....	150
GetAttributes	152
Configure.....	154
Start.....	156
CancelPulse	157
GetInfo	158
7.3 Creating an OPC Server Object	161
Description	161
8. HCI Client Development Guidelines.....	165

8.1 Debugging Your HCI Clients.....	165
How to debug	165
8.2 Quickstart Client – Server Communication.....	165
Permissions and bypassing them.....	165
9. OPC Performance Guidelines.....	167
9.1 Introduction	167
Overview	167
9.2 Achieving Fast Performance with COM.....	167
Comparing In-Process to Local Reals	167
9.3 Summary.....	171
COM rules to follow for optimal performance	171
9.4 Troubleshooting	171
Possible symptoms and solutions	171

Tables

Table 1	Asynchronous Write/OnDataChange Test Groups.....	26
Table 2	Asynchronous Read Test Groups.....	27
Table 3	Asynchronous Refresh Test Groups.....	27
Table 4	Asynchronous Loop Test Groups	28
Table 5	Synchronous Loop Test Groups	28
Table 6	HciTimeout Interfaces and Methods	73
Table 7	AnyTimedCustomMethod Parameter Specifications	81
Table 8	AnyTimedCustomMethod HRESULT Return Codes	82
Table 9	CheckStatus Parameter Specifications.....	84
Table 10	CheckStatus HRESULT Return Codes.....	85
Table 11	AnyTimedAutomationMethod Parameter Specifications	88
Table 12	AnyTimedAutomationMethod HRESULT Return Codes	89
Table 13	Timeout Status Parameter Specifications.....	91
Table 14	GetComponentInfo Parameter Specifications	100
Table 15	GetComponentInfo HRESULT Return Codes.....	100
Table 16	GetCurrentDomainName Parameter Specifications	101
Table 17	GetCurrentDomainName HRESULT Return Codes	101
Table 18	GetLocalHostName Parameter Specifications.....	102
Table 19	GetLocalHostName HRESULT Return Codes.....	102
Table 20	IsLocalHost Parameter Specifications	103
Table 21	IsLocalHost HRESULT Return Codes	103
Table 22	GetComponentInfo Parameter Specifications	105
Table 23	GetComponentInfo HRESULT Return Codes.....	105
Table 24	GetCurrentDomainName Parameter Specifications	106
Table 25	GetCurrentDomainName HRESULT Return Codes	106
Table 26	GetLocalHostName Parameter Specifications.....	107
Table 27	GetLocalHostName HRESULT Return Codes.....	107
Table 28	IsLocalHost Parameter Specifications	108
Table 29	IsLocalHost HRESULT Return Codes	108
Table 30	HresultToString Parameter Specifications.....	109
Table 31	HresultToString HRESULT Return Codes.....	109
Table 32	SetAttributes Parameter Specifications	111
Table 33	SetAttributes HRESULT Return Codes	111
Table 34	SetAttributes ppErrors Return Codes	112
Table 35	GetAttributes Parameter Specifications	113
Table 36	GetAttributes HRESULT Return Codes	114
Table 37	GetAttributes ppErrors Return Codes	114
Table 38	Configure Parameter Specifications	117
Table 39	Configure HRESULT Return Codes	117
Table 40	Start Parameter Specifications	118
Table 41	Start HRESULT Return Codes	118
Table 42	CancelPulse Parameter Specifications.....	119

Table 43	CancelPulse HRESULT Return Codes	119
Table 44	GetInfo Parameter Specifications.....	120
Table 45	GetInfo HRESULT Return Codes.....	121
Table 46	HciSink Parameter Specifications	123
Table 47	Dadvise Parameter Specifications.....	124
Table 48	Dadvise HRESULT Return Codes.....	125
Table 49	Dunadvise HRESULT Return Codes	126
Table 50	SetDataObject Parameter Specifications	128
Table 51	HciStreamUtil Parameter Specifications	131
Table 52	GetByIndex (for OPCSTMFORMATDATETIME) Parameter Specs	133
Table 53	GetByIndex (for OPCSTMFORMATDATETIME) Return Codes.....	133
Table 54	GetByIndex (for OPCSTMFORMATDATA) Parameter Specs.....	134
Table 55	GetByIndex (for OPCSTMFORMATDATA) Return Codes	134
Table 56	GetByIndex (for OPCSTMFORMATWRITECOMPLETE) Param Specs	135
Table 57	GetByIndex (for OPCSTMFORMATWRITECOMPLETE) Return Codes....	135
Table 58	GetByHandle (for OPCSTRMFORMATDATETIME) Param Specs	136
Table 59	GetByHandle (for OPCSTRMFORMATDATETIME) Return Codes.....	136
Table 60	GetByHandle (for OPCSTRMFORMATDATA) Param Specs.....	137
Table 61	GetByHandle (for OPCSTRMFORMATDATA) Return Codes	137
Table 62	GetByHandle (OPCSTRMFORMATWRITECOMPLETE) Param Specs	138
Table 63	GetByHandle (OPCSTRMFORMATWRITECOMPLETE) Return Codes....	138
Table 64	GetGroupHeader Parameter Specifications.....	139
Table 65	GetGroupHeader HRESULT Return Codes.....	139
Table 66	GetItemCount Parameter Specifications	140
Table 67	GetItemCount HRESULT Return Codes	140
Table 68	GetTransID Parameter Specifications.....	141
Table 69	GetTransID HRESULT Return Codes.....	141
Table 70	CreateHciObject Parameter Specifications	145
Table 71	CreateHciObject HRESULT Error Codes.....	145
Table 72	CreateRemoteObject Parameter Specifications.....	146
Table 73	CreateRemoteObject HRESULT Error Codes	146
Table 74	GetComponentInfo Parameter Specifications	147
Table 75	GetComponentInfo HRESULT Error Codes.....	147
Table 76	UTCToLocal Parameter Specifications	148
Table 77	UTCToLocal HRESULT Error Codes	148
Table 78	CheckPoint Parameter Specifications	149
Table 79	CheckPoint HRESULT Error Codes	149
Table 80	SetAttributes Parameter Specifications.....	150
Table 81	SetAttributes HRESULT Return Codes.....	150
Table 82	SetAttributes ppErrors Return Codes.....	151
Table 83	GetAttributes Parameter Specifications	152
Table 84	GetAttributes HRESULT Error Codes	152
Table 85	GetAttributes ppErrors Return Codes	153

Contents

Table 86	Configure Parameter Specifications	154
Table 87	Configure HRESULT Error Codes	155
Table 88	Start Parameter Specifications	156
Table 89	Start HRESULT Error Codes	156
Table 90	CancelPulse Parameter Specifications.....	157
Table 91	CancelPulse HRESULT Return Codes.....	157
Table 92	GetInfo Parameter Specifications	158
Table 93	GetInfo HRESULT Return Codes	159

Figures

Figure 1	TPS Directory Structure	22
Figure 2	Non-Managed Nodes.....	38
Figure 3	HCI OutOfProc OPC Server (EXE).....	52
Figure 4	Internal Functionality of the HciTimeout Object.....	72
Figure 5	Comparing Group Size vs. Sync Read Time	168
Figure 6	Comparing Device Reads vs. Cache Reads	169
Figure 7	Viewing Group Active State	170

1. Getting Started

1.1 Introduction

Purpose

This guide describes tools, techniques, and procedures of interest to those users tasked with the development of Honeywell Communications Interface (HCI) client applications. It describes the facilities available to the client application developer and the Application Program Interface (API) for client applications.

It is assumed that the reader is familiar with C++ or Visual Basic, and is engaged in the development of HCI client application software.

Abstract

The standardized blueprint for enabling disparate industrial automation manufacturing systems to work together is through a new Component Object Model (COM) specification called "OLE for Process Control" or "OPC." Honeywell is a member of the OPC consortium and is leading the way in the application of the standard. Honeywell's current development of HCI Client products is gaining the attention of end-users and vendors, proving that HCI and OPC compliant components will be a required element of future industrial software applications.

The OPC Specifications are maintained by the OPC Foundation. HCI implements the OPC initiatives for Data Access and Security. OPC defines a set of custom COM interfaces and methods, for developers to use in building their industrial automation software applications.

The driving force for the power and appeal of the OPC standard is Microsoft's OLE and COM technology. Microsoft is now concentrating their efforts on enterprise-wide computing with a client/server strategy that is based on the company's Windows 2000/2003 operating system, COM and Microsoft's Back Office suite of products. Windows 2000/2003 provides the operating system foundation on which the enterprise-wide strategy is built. COM is a set of protocol interfaces for objects that enables one application to use the features of any available COM object, which supports the desired interface.

1 Getting Started

1.1 Introduction

Honeywell Communications Interface further enhances this object communication capability. The primary purpose of HCI is to support interoperability among TPS clients and servers. The overall approach for the TPS system is compliant with OPC. It relies on COM to provide the basis for distributed object communication and object management. Distributed COM (DCOM) provides the underlying transport mechanisms that are used to meet the inter-object communications requirements of the TPS system. This includes such items as transparent distributed object communications, server activation security, optional call (method) security, concurrency support, and release independence through immutable interfaces. DCOM is the communication mechanism used between TPS clients and TPS servers.

Features

This document provides information for implementing HCI client applications. The contents of this document include:

- Guideline for installing the HCI client library and runtime on your system
- Description of HCI and OPC concepts
- Examples showing basic guidelines when using the custom and automation interfaces
- Description of the custom and automation APIs provided for HCI client applications

OPC Data Access 2.05 custom interfaces

IOPCCommon	IOPCGroupStateMgt
IConnectionPointContainer (Server)	IOPCServer
IOPCSyncIO	IOPCItemProperties
IOPCItemMgt	IOPCBrowseServerAddressSpace
IOPCAsyncIO2	IOPCAsyncIO (OPC 1.0A only)
IEnumOPItemAttributes	IConnectionPointContainer (Group)
IDataObject (OPC 1.0A only)	

OPC Data Access 1.0A automation interfaces

IOPCServerDisp	IOPCGroupStateMgtDisp
IOPCSyncIODisp	IOPCAsyncIODisp
IOPCItemMgtDisp	IOPCItemDisp

OPC Data Access 2.02 automation objects

OPCServer	OPCGroups
OPCGroup	OPCItems
OPCItem	OPCBrowser

OPC Security 1.0 custom interface

IOPCSecurityNT

1 Getting Started

1.2 Requirements

1.2 Requirements

Target audience and prerequisite skill recommendations

The purpose of this guide is to provide developers with basic reference material for development of HCI client applications for the TPS system. It is assumed that you are familiar with Microsoft COM technology, OPC specifications, and C++ or Visual Basic.

Operating system

OPC and HCI Clients can run on Windows 2000 Professional or Server, Windows XP or Windows 2003 server, within a TPS domain, or outside a TPS domain.

HCI client applications can be written using Visual C++, Visual Basic, or Visual Basic scripts for applications such as Microsoft Excel.

Hardware environment

The following table lists the minimum PC hardware requirements for the HCI Client software.

Item	Requirement
CPU	Intel 200 MHz (or higher) Pentium Pro Intel 133 MHz Pentium (minimum)
Memory	64 MB EDO ECC standard, expandable to 256 MB
Hard Disk	2 GB standard, expandable to 9.3 GB

Software environment

This table lists the Windows operating system, Visual C++ and Visual Basic versions that have been validated to run with HCI Client software.

Verify that the client node has been upgraded to Windows 2000 or Windows XP/2003 with the latest hot fixes and Microsoft service packs supplied by Honeywell.

Item	Requirement
Operating System	Windows 2000 Windows 2000 Professional or Server Windows XP/2003 Windows XP or Windows 2003 Server Standard Edition
Visual C++	VC++ 6.0 (including Service Pack 2)
Visual Basic	VB 6.0

Network environment

This table lists the network types, protocols, application layer software, and name services that have been validated to run with HCI Client software.

Item	Requirement
PIN/PCN	Ethernet, TCP/IP, Microsoft application layer software such as RPC, DCOM. One of the following name services must be available: • DNS • WINS • Local Host File

External references

Inside OLE, Second Edition, May, 1995, Kraig Brockschmidt, Microsoft Press

Component Object Model Specification, Draft Version 0.9, October 24, 1995, Microsoft Corp. and Digital Equipment Corp. (available from Microsoft's FTP site)

OLE for Process Control Specification (OPC), Sept. 11, 1997, Final Version 1.0A

OLE Automation Programming Reference, Microsoft Press, Redmond, WA, 1996

OLE 2 Programming Reference, Vol. 1, Microsoft Press, Redmond, WA, 1994

1 Getting Started

1.2 Requirements

About this guide

This guide is organized into the following major sections. It also has an extensive glossary for a quick reference to acronyms, terminology, and definitions.

- **Installation Verification.** Explains how the HCI runtime and client toolkit are installed. It may be skipped if the HCI is already installed on your system.
- **Key Concepts.** Presents a brief overview of the TPS system, HCI and OPC.
- **Developing an HCI Client Application.** Provides information on the development environment including naming conventions and standards, and how to determine whether to use the custom or automation interface for client applications.
- **HciTimeout Object.** Provides information regarding the object's ability to invoke supported OPC interface methods.
- **Custom Interface APIs.** Describes the Custom Interface API for C++ applications including the OPC object interfaces and HCI client library classes.
- **Automation Interface APIs.** Describes the Automation Interface API for VB and scripting applications including OPC object interfaces and HCI VB helper classes.
- **HCI Client Development Guidelines.** Provides guidelines for debugging your HCI clients and doing a quickstart for client/server communication.
- **OPC Performance Guides.** Contains information about optimizing performance with COM. There is also a troubleshooting table for resolving client implementation error symptoms.

2. Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

This section is of interest primarily to the system administrator or to anyone who needs to verify the installation of the HCI software package on their system. If HCI is already installed and functioning on your system, you may skip this section.



ATTENTION

The intent of this section is to provide a method of verifying the installation of the HCI and Toolkit software. The *TPS System Implementation Guide* provides complete instructions for installing [all](#) of the TPS software. This includes the HCI and Toolkit software package according to the licensing agreement. Please consult that publication for installation-related tasks.

Overview

The HCI software consists of a set of two installable TPS package components: HCI Runtime and HCI Client Toolkit. The HCI Client Toolkit may be installed on both TPS nodes and non-TPS nodes. Base node software is not required to develop clients. However, you would need the base node software if you want to execute the target server within TPS System Management (such as, to view the server from the Honeywell System Management display). Installation of the toolkit on a non-TPS node does require additional steps after the installation procedure. Refer to the section, Accessing HCI Server for a description of these steps.

The HCI software package and other TPS package(s) can be uninstalled. The uninstall procedure is covered in the *Software Release Notice (SCN)*.



REFERENCE - INTERNAL

For complete details on prerequisites and installing the TPS software, refer to the *TPS System Implementation Guide*

2 Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

Verification of installed directories

Upon successful installation, your selected drive directory should include a folder called TPS containing the folders and sub-folder structure similar to what is shown in the following figure.

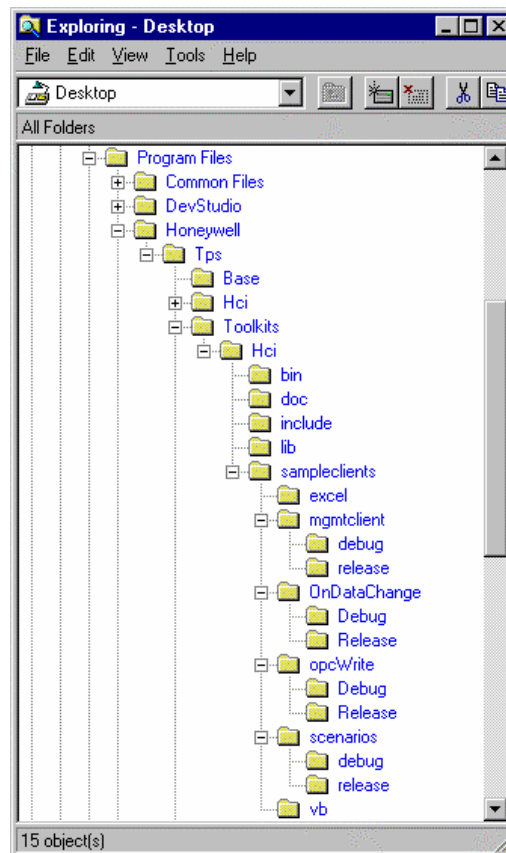
The HCI subdirectory contains the HCI Runtime software. The Toolkits folder contains header files that could be used by the HCI clients.



ATTENTION

The content of the TPS folder will vary depending on what kind of node is being used and what software packages are being installed.

Figure 1 TPS Directory Structure



**ATTENTION**

If the HCI Client Development Library was not selected, subdirectory `\ProgramFiles\Honeywell\TPS\Toolkits` will not exist.

**ATTENTION**

In order for an HCI client to remotely access an HCI component, the client connectivity package for that component must be installed locally on the client node. For example, if the HCI client intends to connect remotely to the TPN Server, then you are required to install the TPN Server client connectivity package on the same TPS node or non-TPS node that the HCI client resides.

Subdirectory summary**`\ProgramFiles\Honeywell\TPS\Base`**

The TPS Base Infrastructure component set contains the basic functionality required for almost all other TPS components. Base software functionality includes TPS system administration, event handling, registry access, and versioning control, etc. Also included in Base are Microsoft “redistributable” DLLs upon which many TPS components depend. As part of the HCI software package installation, the TPS node’s global “Path” environment variable is modified to include the path to Base.

Note that, for the most part, the files contained within the Base directory are created during installation of the TPS Base Infrastructure Component set, which must be accomplished before the HCI Runtime is invoked. It is mentioned here because the HCI Runtime may be *installed* (though not *invoked*) prior to the TPS Base Infrastructure. In addition, since a small part of the HCI Runtime resides within TPS Base, the `\ProgramFiles\Honeywell\TPS\Base` subdirectory will be created.

2 Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

\\ProgramFiles\\Honeywell\\TPS\\Hci

Almost all of the functionality provided by the Honeywell Communications Interface is contained within the DLLs and executables of this directory and substantially represents the whole of the HCI Runtime. Unlike the Base, the path to this directory is not added to the global “Path” environment variable, as the files contained within this directory are self-registration capable. By omitting this path and being reliant on the unique registry entries that point to these files, HCI Clients and Servers are assured of running only with the versions of HCI Runtime components that they expect.

\\ProgramFiles\\Honeywell\\TPS\\Toolkits\\Hci

The HCI Toolkit subdirectory contains header files (include) necessary for compiling and libraries (lib) necessary for linking clients. Additionally, this subdirectory contains debug versions of relevant Base and HCI Runtime DLLs (bin), necessary to support debug-targeted builds of clients.

The HCI Toolkit subdirectory also contains a variety of sample clients which, when built, demonstrate the steps that the client author follows when creating custom clients using the HCI Development Libraries.

Within the HCI client toolkit, the source code for these applications is also provided to aid you in writing HCI clients. You can run the compiled version of these applications by either double-clicking on their icon or by dragging them to the Start\\Run dialog box. The sample clients included in the HCI Client Toolkit are:

- **opc.xls:** This is a VB script application for MS Excel. This simple client connects to the HciDssTemplate sample server “template_server1,” creates a group (AddGroup), adds 4 items to the group (AddItems), activates the group, and then performs a read from cache (Read). This all takes place when the CommandButton1 is clicked from the MS Excel sheet. You should notice that the timestamps update with UTC time. The source code is available for review or modification. It is located in the excel subdirectory under the sampleclients directory. Opc95.xls is a version compatible with Microsoft Excel 95. It performs the same functions as opc.xls. Both opc95.xls and opc.xls are OPC 2.0 and OPC 1.0A compliant clients.
- **opc.bas:** This is a Visual Basic 6.0 ActiveX executable application that connects to a server, such as the HciDssTemplate sample server “template_server1”. A different server (component name) can be chosen. The client will then create a group (AddGroup) and add items to the group (AddItems). The items can either come from a file which must be named group1.txt and reside in c:\\temp, or if this file does not exist, then 10 points will be generated that correspond to the sample server. The group will then establish an asynchronous call back for onDataChange notices. The group is then activated and will listen for onDataChange. A manual Refresh button

is available to cause an OnDataChange. The client will continue to listen until it is shutdown. Opc.bas is an OPC 1.0A compliant client.

- **mgmtclient.exe:** This application takes the HCI server name as a command line argument. If none is provided, the application will run against the HciDssTemplate server. This application exercises synchronous and asynchronous reads, as well as several HCI component management functions. For example, through this sample client you can start/stop the server, get its status, get an interface pointer to the reserved instance within the server, or enumerate all instances within the server. This application prompts you before every test to allow you to selectively choose or repeat the desired tests. Mgmtclient.exe is an OPC 1.0A compliant client.
- **OnDataChange.exe:** This application connects to the HciDssTemplate sample server "template_server1." The client will then create a group (AddGroup) and add items to the group (AddItems). The items can either come from a file which must be named group1.txt and reside in c:\temp, or if this file does not exist, then 10 points will be generated that correspond to the sample server. The group will then setup an IAdviseSink using the hciSink. This client will then listen for OnDataChanges. For each OnDataChange notification that is received, a display will be shown of the new values. After 25 notifications have been received, the client will shutdown. OnDataChange.exe is an OPC 2.0 compliant client.
- **opcWrite.exe:** This application connects to the HciDssTemplate sample server "template_server1". The client will then create a group (AddGroup), add items to the group (AddItems). The items can either come from a file which must be named group1.txt and reside in c:\temp, or if this file does not exist then 10 points will be generated that correspond to the sample server. The group will then write the points values to the server. These points are changed and written to the server 1000 times. After 1000 Writes have been performed, the client will shutdown. OpcWrite.exe is an OPC 1.0A and OPC 2.0 compliant client.
- **CopClient.exe:** This application provides a GUI that allows you to select an OPC server with which to communicate. If the server resides on a remote node, then the node name can be used to access that server. The purpose of the client is to illustrate the usage of OPC security. Based on radio buttons on the user interface, the client can be set to use no OPC security, OPC Security based on the current Signon Manager (COP) user, or OPC Security using text credentials collected from the user interface. Note that based on COM restrictions, the text credentials can apply only to servers on remote nodes. When the Read button is clicked, the client attempts to read the points specified in the Point.Parameter file, using the selected security method and credentials, and reports the results in the edit box.
- **Scenarios Client:** This application provides a GUI that allows you to select an OPC server in which to communicate. If the server resides on a remote node, then the

2 Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

node name can be used to access that server. Either threading model of this client can be selected, free-threaded or apartment model threaded. If you choose to continue on errors, then most errors will be ignored, attempting to proceed on errors. The client will repeat its testing in a loop depending on the number of iterations chosen. If forever is chosen, then the client will run in an infinite loop. There are several different scenarios that can be run (some scenarios use OPC 1.0A methods while other scenarios use OPC 2.0 methods):

- Asynch Write/OnDataChange test will perform Asynchronous writes in a loop. The data will change for each write. In addition, an OnDataChange will be active for each group. There are 6 groups with the following setup:

Table 1 Asynchronous Write/OnDataChange Test Groups

Group	Active	NumItems	Status
1	Yes	480	changing
2	Yes	280	changing
3	Yes	200	changing
4	Yes	40	changing
5	Yes	11	changing
6	Yes	10	not changing

The items are read from one of two places. If the file for each group exists, then the items are read from it (Examples: c:/temp/group1.txt, c:/temp/group2.txt, and so forth). If these files do not exist, the points are generated as p0.pv, p1.pv, and so forth, with the datatype as VT_EMPTY.

- Asynch Read test will perform Asynchronous reads in a loop. There are 6 groups with the following setup:

Table 2 Asynchronous Read Test Groups

Group	Active	NumItems	Status
1	No	480	not changing
2	No	280	not changing
3	No	200	not changing
4	No	40	not changing
5	No	11	not changing
6	No	10	not changing

The items are read from one of two places. If the file for each group exists, then the items are read from it (Examples: c:/temp/group1.txt, c:/temp/group2.txt, and so forth). If these files do not exist, then the points are generated as p0.pv, p1.pv, and so forth, with the datatype as VT_EMPTY.

- Asynch Refresh will perform Asynchronous Refresh in a loop. There are 6 groups with the following setup:

Table 3 Asynchronous Refresh Test Groups

Group	Active	NumItems	Status
1	No	480	not changing
2	No	280	not changing
3	No	200	not changing
4	No	40	not changing
5	No	11	not changing
6	No	10	not changing

The items are read from one of two places. If the file for each group exists, then the items are read from it (Examples: c:/temp/group1.txt, c:/temp/group2.txt, and so forth). If these files do not exist, the points are generated as p0.pv, p1.pv, and so forth, with the datatype as VT_EMPTY.

2 Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

- Asynchronous Loop test will perform Asynchronous writes in a 5 second loop on 13 inactive groups, each containing 1 float. There are 16 groups with the following setup:

Table 4 Asynchronous Loop Test Groups

Group	Active	Update Rate	NumItems	Status
1	Yes	1	13	changing
2	Yes	2	104	changing
3	Yes	2	26	changing
4-16	No	N/A	1	not changing

The items are read from one of two places. If the file for each group exists, then the items are read from it (Examples: c:/temp/group1.txt, c:/temp/group2.txt, and so forth). If these files do not exist, the points are generated as p0.pv, p1.pv, and so forth, with the datatype as VT_EMPTY.

- Synchronous Loop test will perform Synchronous read/write in a loop on 4 inactive groups. There are 8 groups with the following setup:

Table 5 Synchronous Loop Test Groups

Group	Active	NumItems	Status
1	No	1	not changing
2	Yes	50	changing
3	Yes	28	changing
4	Yes	13	changing
5	No	13	not changing
6	No	28	not changing
7	No	60	not changing
8	No	1	not changing

The items are read from one of two places. If the file for each group exists, then the items are read from it (Examples: c:/temp/group1.txt, c:/temp/group2.txt, and so forth). If these files do not exist, the points are generated as p0.pv, p1.pv and so forth, with the datatype as VT_EMPTY.

- HciClient test will perform some basic HCI client tests. These tests include creating one group and doing basic synchronous/asynchronous calls and displaying the results. The items are read from one of two places. If the file for each group exists, then the items are read from there (Example: c:/temp//group1.txt). If this file does not exist, then the points are generated as p0.pv, p1.pv, and so forth, with the datatype as VT_EMPTY. The client does some basic dsstemplate tests of the IWhatever interfaces and the Attributes.
- Timeout test will perform several OPC methods using the HCI Timeout interface. For this test, the number of iterations is used as the timeout value in milliseconds. If forever is chosen, then the timeout interface will not be used.

2 Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

`\\ProgramFiles\\Honeywell\\TPS\\Toolkits\\Hci\\Sampleservers`

The HCI Client Toolkit also contains a sample server for use in exercising the HCI sample clients described above, or your own clients. The sample server included in the HCI Client Toolkit is:

- **dsstemplate.exe, dsstemplate.dll:** This application provides a sample implementation of an HCI Specific Server which supports both OPC 1.0A and OPC 2.0. The sample implementation simulates access to a control system. A list of itemIDs to be known by the server is read from an input file when the server starts up. An array of these items including the canonical data type, current value, timestamp, and quality is kept in memory. The initial item values are read from the file. This value may be read and written by both OPC 1.0A and 2.0 clients through the OPC interfaces. The "database" is check-pointed to different files when the server shuts down.
- **template_server1.dss:** The input file is the sample server database. Item IDs for the sample server are the index to the item in the database as entered. If items are added to the file when a checkpoint exists, they must be added to the end of the file. If items are inserted, then the existing checkpoint file must be deleted. Failure to do this results in a corrupted checkpoint file.

The installation of the sample server includes its HCI component configuration. This configuration is minimal in that it is based primarily on default values. However, this default configuration is more than sufficient for clients to connect to the sample server and make OPC requests.

For clients that want to override any default configuration values, the sample server configuration can be changed using the HCI Component Configuration page. The HCI Component Configuration operates within the System Management Display implemented as a Microsoft Management Console (MMC) Snap-in.

You must install the sample servers on the desired TPS nodes before configuring. The sample server is defined as a "local component." This means that the server is accessed only from its local node. The configuration, including its alias (component name) is the same on all nodes on which the sample server has been installed. Any changes to the configuration are replicated to all nodes on which the sample server has been installed.

Procedure to configure the HCI sample server

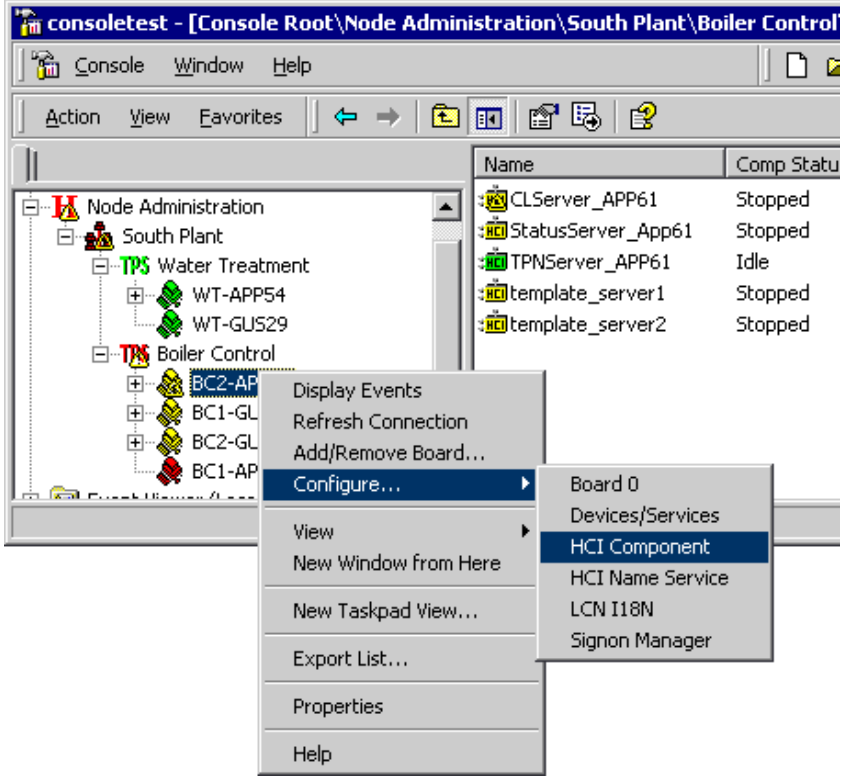
Here are the steps required to configure the HCI sample server (template_server1).

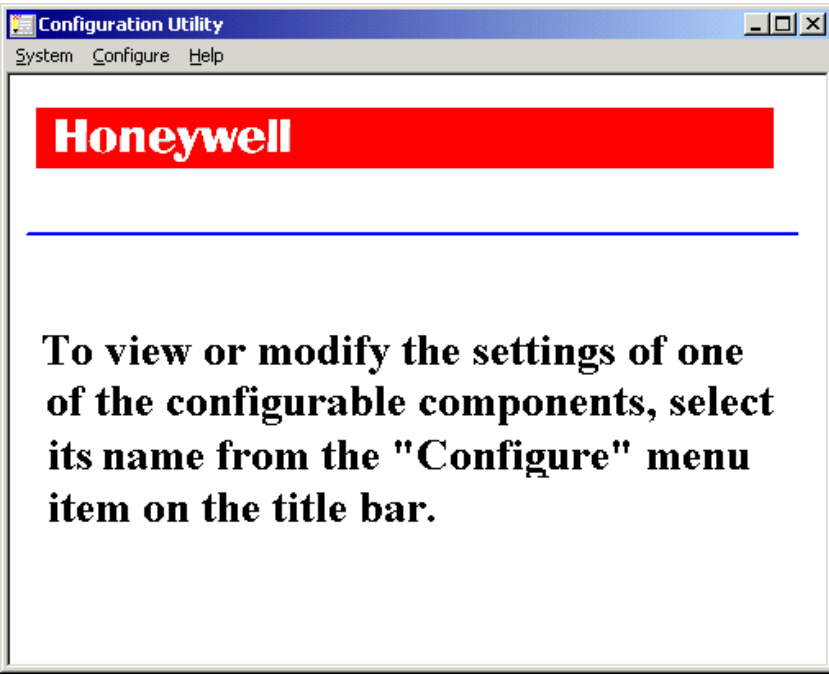
Before You Start! If you have the optional Honeywell Node Administration Snap-in loaded on the PC that you are using, start with **Step 1**. If you do not, start with **Step 4**.

Step	Action
	Important: The first three steps in this procedure assume you have already loaded the Honeywell Node Administration Snap-in on the PC that you are using, and that you have the MMC running. If you do not have the Snap-in loaded on the PC, skip to Step 4. REFERENCE - INTERNAL If you need help with the MMC or loading the Honeywell Node Administration Snap-in, refer to the section <i>Configuring a System Management Display in the System Management Configuration Guide</i>.
1	Right click the node name . (In the example that follows, APP node: BC2-APP61 in Boiler Control is selected.)

2 Installation Verification

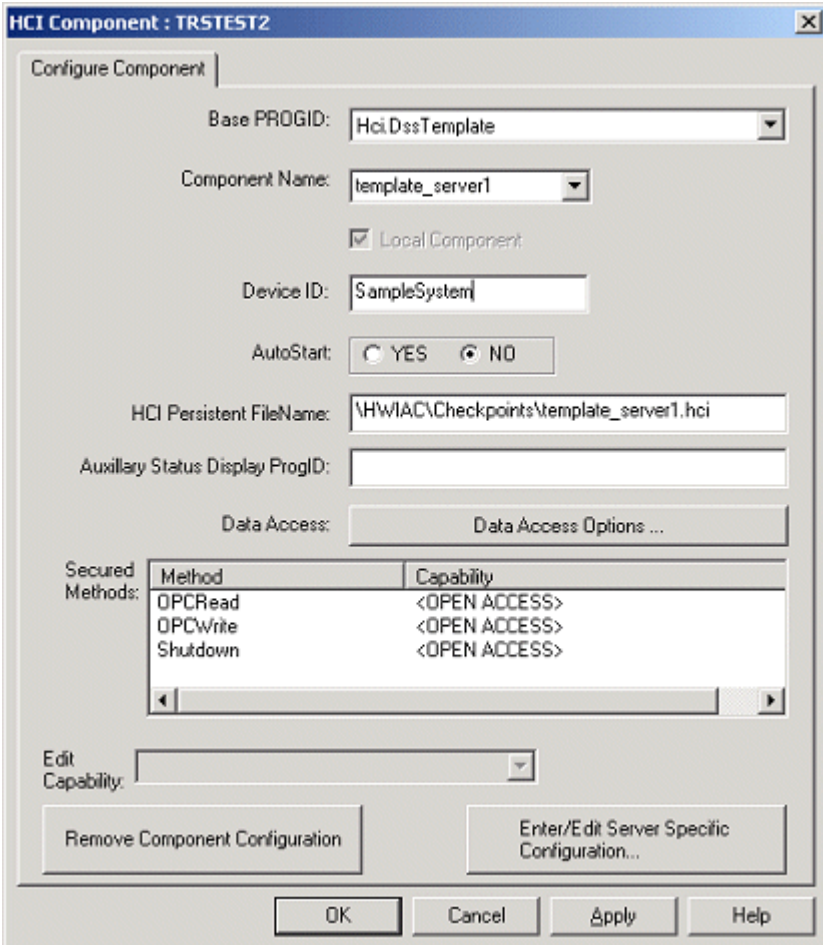
2.1 Verifying the HCI Client Developer Toolkit Installation

Step	Action												
2	Select Configure... from the popup context menu, then select HCI Component from the next popup context menu.  The screenshot shows the 'consoletest' application window. The title bar reads 'consoletest - [Console Root\Node Administration\South Plant\Boiler Control]'. The menu bar includes 'Console', 'Window', and 'Help'. Below the menu bar is a toolbar with icons for 'Action', 'View', 'Favorites', and navigation. The main area is divided into a tree view on the left and a list view on the right. The tree view shows a hierarchy: 'Node Administration' > 'South Plant' > 'Water Treatment' > 'Boiler Control'. The 'Boiler Control' folder is expanded, showing sub-items: 'BC2-APP', 'BC1-GL', 'BC2-GL', and 'BC1-AP'. The 'Boiler Control' folder is selected. The list view on the right shows a table with columns 'Name' and 'Comp Statu'. The table contains the following data: <table border="1"> <thead> <tr> <th>Name</th> <th>Comp Statu</th> </tr> </thead> <tbody> <tr> <td>CLServer_APP61</td> <td>Stopped</td> </tr> <tr> <td>StatusServer_App61</td> <td>Stopped</td> </tr> <tr> <td>TPNServer_APP61</td> <td>Idle</td> </tr> <tr> <td>template_server1</td> <td>Stopped</td> </tr> <tr> <td>template_server2</td> <td>Stopped</td> </tr> </tbody> </table> A context menu is open over the 'Boiler Control' folder. The menu items are: 'Display Events', 'Refresh Connection', 'Add/Remove Board...', 'Configure...', 'View', 'New Window from Here', 'New Taskpad View...', 'Export List...', 'Properties', and 'Help'. The 'Configure...' item is selected. A second context menu is open over the 'Configure...' item. The menu items are: 'Board 0', 'Devices/Services', 'HCI Component', 'HCI Name Service', 'LCN I18N', and 'Signon Manager'. The 'HCI Component' item is selected.	Name	Comp Statu	CLServer_APP61	Stopped	StatusServer_App61	Stopped	TPNServer_APP61	Idle	template_server1	Stopped	template_server2	Stopped
Name	Comp Statu												
CLServer_APP61	Stopped												
StatusServer_App61	Stopped												
TPNServer_APP61	Idle												
template_server1	Stopped												
template_server2	Stopped												
3	Continue with Step 6 .												

Step	Action
4	If you are starting the procedure to configure the HCI sample server, and you haven't performed steps 1 through 3, Click Start → Programs → Honeywell TPS → Configuration Utility Result: The Configuration Utility screen appears. 
5	Click Configure , and select HCI Component from the drop-down list.
	ATTENTION In order to connect to HCI components without the use of the HCI helper functions (GetComponentInfo), the client will require each component's ProgID. The ProgID for the Honeywell sample server is Hci.DssTemplate.

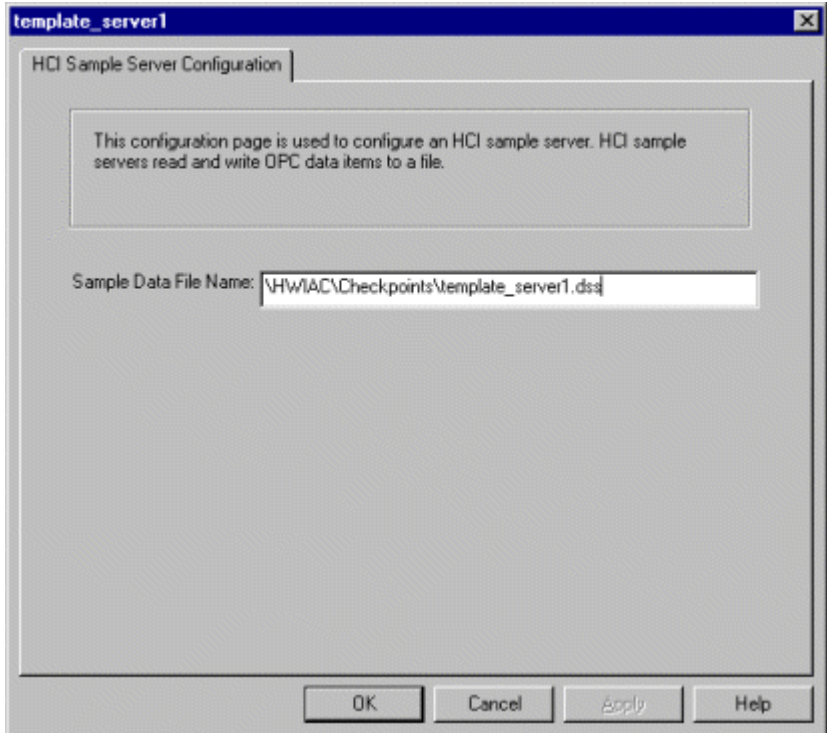
2 Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

Step	Action
6	Since a configuration for the sample server already exists as a result of installation, the sample server component name will appear in the drop-down list for the Component Name: field when clicked. Select "template_server1" from the Component Name: drop-down list. Results: 

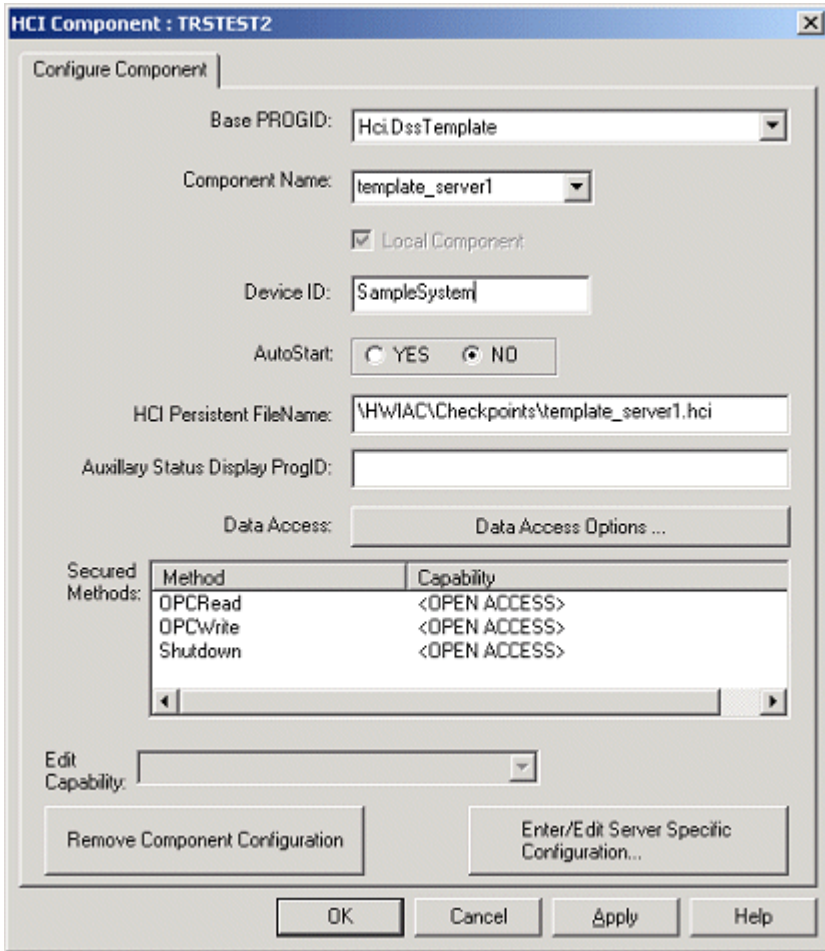
2 Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

Step	Action
7	This HCI sample server has all the basic HCI component configuration fields, as well as items specific to the HCI sample server. To access the specific items, click the "Enter/Edit Server Specific Configuration..." button.
8	The Sample Data File Name and location defaults to \HWIAC\Checkpoints\<component_name>.dss. If using this default location, ensure the path and sample file name is: \HWIAC\Checkpoints\template_server1.dss, and click OK. 

2 Installation Verification

2.1 Verifying the HCI Client Developer Toolkit Installation

Step	Action
9	Make any configuration changes if needed, then click OK.  Results: This copies your configuration to all TPS nodes on which the sample server has been installed.

**REFERENCE - INTERNAL**

For complete Domain configuration instructions, see the section named TPS Domains and Consoles as OUs in the *System Management Configuration Guide*

**ATTENTION**

If you choose to change the name of the sample input data file, you must manually create the file with the name and location specified on the configuration page. Failure to create the sample input file results in a warning state from your sample server during startup.

Other system updates**Registry**

As previously indicated, the HCI Runtime binaries that reside within the \ProgramFiles\Honeywell\TPS\Hci subdirectory are self-registration capable. In other words, when invoked as a parameter to “regsvr32.exe,” these objects will identify themselves at some known location in the system registry. COM objects that, for the most part, define the HCI Runtime, enter their information by CLSID (class identifier) and ProgID (programmatic identifier) under registry hive HKEY_CLASSES_ROOT.

As the installation proceeds, a built-in feature of the installation script, regsvr32 is invoked for each of the COM objects in the HCI Runtime. Once the script has finished the installation, the HCI Runtime is fully registered under HKEY_CLASSES_ROOT.

Self-registration output from the remaining non-COM pieces of the HCI Runtime (For example, Event Log identifiers, sample client/server configuration information) can be found under “HKEY_LOCAL_MACHINE\Software\Honeywell.”

System Environment

Most of the HCI Runtime is dependent upon Base for common infrastructure facilities. Because most of Base is not self-registration capable, the location of Base must be made known system-wide. Therefore, as part of the package installation, the system Path environment variable is updated to include the path to Base.

If the client libraries are installed, the value <drive:>\ProgramFiles\Honeywell\TPS\Toolkits\Hci\bin is also added to the Path environment variable (Remember that this directory contains debug DLLs necessary for the execution of clients/servers built to the debug target).

2 Installation Verification

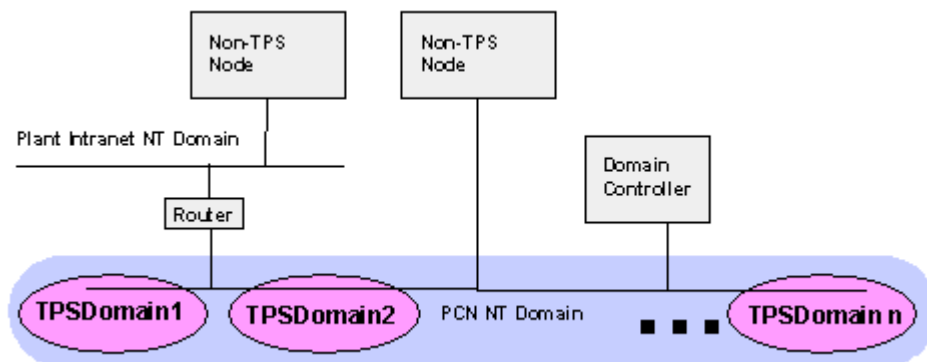
2.2 Developing HCI Clients on Non-TPS Nodes

2.2 Developing HCI Clients on Non-TPS Nodes

Overview

A non-managed node is a node that resides outside of a TPS domain. Non-managed nodes may exist on the same Windows 2000/2003 domain containing the TPS domain, or on a Windows 2000/2003 domain that is separate from the TPS Domain's Windows 2000/2003 domain, as shown in the following drawing.

Figure 2 Non-Managed Nodes



For client development, a non-managed node may also be a stand-alone node that is completely isolated from any TPS domain. To facilitate this capability, the non-managed node must be properly configured after you have properly installed the Client Toolkit package.

The HCI Client toolkit includes a sample server to facilitate your testing. This server is named "Hcidsstemplate.exe" and is located at <install drive:>\Program Files\Honeywell\TPS\HCI. This server is registered when the toolkit is installed.

The rest of this section identifies the remaining steps *after* you have installed the HCI Client Toolkit, which will allow you to develop and test clients on a non-TPS node.

Procedures to configure a non-TPS node for HCI client development

What follows is a sequence of procedures that cumulatively configure a non-TPS node for HCI client development. Please note there are a couple of them that are optional.

Verify the identity setting of HciDssTemplate, as follows:

Step	Action
1	Invoke dcomcnfg tool.
2	Double-click the entry, Hci.DssTemplate – A Sample HciComponent – exe Server .
3	Select the Identity tab.
4	Verify that the 1 st radio button, The interactive user , is selected.
5	Click Apply , click OK , then click OK again to exit dcomcnfg.

Modify the registry to allow the sample server to run outside a TPS domain, as follows:

Step	Action
1	Invoke the regedt32 tool.
2	Go to the registry window, HKEY_LOCAL_MACHINE on Local Machine
3	Browse to the following path: SOFTWARE/Honeywell/MyTpsDomain/HciComponents then open HciComponents to find the entry for template_server1 .
4	Double-click template_server1 to view its current configuration. NOTE: If the configuration does not show up in a frame on right side, use the View menu, and select Tree and Data .
5	Click template_server1 (in left frame). From the Edit menu, select Add Value... This will invoke the Add Value dialog box.
6	Enter the string TPSManaged in the Value Name field and select OK . This will bring up a second dialog box, String Editor.
7	Enter the string "ByPassSSM" in the String field and select OK . You will now see your new value in the right frame of the window.

2 Installation Verification

2.2 Developing HCI Clients on Non-TPS Nodes

Optional Procedure: Set Live Data option:



ATTENTION

Perform the next three steps **ONLY** if you want the real data (Variant type VT_R4) to change every second within the server.

Step	Action
1	Click template_server1 (in left frame). From the Edit pull-down menu, select Add Value... This will invoke the Add Value dialog box.
2	Enter the string " LiveData " in the Value Name field and click OK . This will bring up a second dialog box, String Editor.
3	Enter the string " yes " in the String field and select OK . You will now see your new value in the right frame of the window. You can turn the live data off by setting this value to "no".

Optional procedure – Simulate device access time option:



ATTENTION

Perform the next four steps if you want to simulate device access time, or slow the server down while performing Reads or Writes.

Step	Action
1	Perform the steps in "Set Live Data Option" in the previous procedure.
2	Click template_server1 (in left frame). From the Edit pull-down menu, select "Add Value..." This will invoke the Add Value dialog box.
3	Enter the string " Delay " in the Value Name field and click OK . This will bring up a second dialog box, String Editor.

2 Installation Verification

2.2 Developing HCI Clients on Non-TPS Nodes

Step	Action
4	Enter the string "0" in the String field and select OK. You will now see your new value in the right frame of the window. The value "0" indicates no delay. Edit this value to adjust the delay time. The value is in milliseconds, (For example, 1000 equals 1000 milliseconds or 1 second.)

Verify that the server will run standalone:

Step	Action
1	From Windows Explorer, double-click the sample server: <install drive:>\Program Files\Honeywell\TPS\HCI\hcidssstemplate.exe
2	Answer "Yes" to run this server.
3	Validate that the server continues to run by invoking the Windows Task Manager. Look in the process list for hcidssstemplate.
4	If not running, check the Windows Event Viewer for error messages. It may be necessary to adjust the launch permissions or access permissions under default security on dcomcnfg.

2 Installation Verification

2.2 Developing HCI Clients on Non-TPS Nodes

Test server with sample client:

	ATTENTION Use the VB client located at <install drive:>\Program Files\Honeywell\TPS\HCI\opc.exe to run against template_server1.
---	---

Step	Action
1	From Windows Explorer, double-click on the sample client: <install drive:>\Program Files\Honeywell\TPS\HCI\opc.exe
2	Press the Start button. It will connect to the sample server and continue to receive data if the LiveData value is set to "yes." Results: The data points of the server are read in from a file. The file is <install drive:>\Hwiac\Checkpoints\template_server1.dss. This file can be modified to contain any custom point that you want the server to be aware of. Just edit the file and add points using the same format that is in the file. This format includes a point name, data type (VARIANT), OPC quality, and value.

You should now be able to connect your custom client to our HCI sample server. Use the sample clients as examples of how to connect to template_server1.

2.3 Accessing HCI Server From Remote Client

Client considerations

For the purpose of this discussion, a Remote Client is a node that resides outside of a TPS Domain. A Remote Client may exist on the Windows domain that contains the TPS Domain or on a separate Windows domain. Also, for the purpose of this discussion, an HCI client is an OPC client, but it is one that is built with the HCI Client Toolkit. In addition to the OPC interfaces, the HCI client may use any of the added value HCI features such as timeouts or server location facilities. Use of the server location feature may require additional client node configuration.

Procedure to set up client application access to remote HCI servers

The following procedures may be required in order to allow HCI or OPC client applications to access remote HCI servers residing within a TPS Domain.

Step	Action	Comments
1 	If the client and server nodes are in two different Windows 2000/2003 domains, then a trust relationship must be established. REFERENCE – INTERNAL For more information on the trust relationships, see the section called <i>Two-way trust relationships</i> in the <i>TPS System Implementation Guide</i>	Necessary for both OPC and HCI clients. A two-way trust may be required if using OnDataChange or Asynchronous IO.
2 	Apply security if you want users in other domains to be able to access local resources. REFERENCE – INTERNAL See the <i>Configure HCI/OPC Client Node</i> and <i>TPS Security</i> sections in the <i>TPS System Implementation Guide</i> and the <i>TPS System Administration Guide</i> for more information.	
3 	Install the System Management Runtime software. REFERENCE – INTERNAL For the procedure to install this software, see the <i>Install System Management Runtime Package</i> section in the <i>TPS System Implementation Guide</i>	This is required for HCI clients using IHciClientUtil::GetComponentInfo.

2 Installation Verification

2.3 Accessing HCI Server From Remote Client

Step	Action	Comments
4	Install OPC server-specific client connection software. For the TPN Server, install TPN Server Client Connectivity and Rebase software.  REFERENCE – INTERNAL If installing TPN Server software, refer to the section <i>Install HCI Client Connectivity Software</i> in the <i>TPS System Implementation Guide</i>.	
5	Enable the HCI Server Node to accept communications from the Client Node.  REFERENCE – INTERNAL For detailed instructions, refer to the section <i>Enable HCI Server to Accept Communications from HCI/OPC Client</i> in the <i>TPS System Implementation Guide</i>.	Necessary for both HCI and OPC Clients.

2 Installation Verification

2.3 Accessing HCI Server From Remote Client

Step	Action	Comments
6 	Generate HCI Server Component Name File. REFERENCE – INTERNAL <i>For details, see the Multicast Scope Purpose section in the System Management Configuration Guide and the Generate HCI Server Component Name File section in the TPS System Implementation Guide.</i>	Necessary for HCI clients that have utilized IHciClientUtil::GetComponentInfo and are not in the same multicast scope.

2.4 Custom Client Installation

Overview

This section is of interest to those users who have developed an HCI client, which needs to be installed in the TPS system. If there are no custom HCI clients to install, you may skip this section.

Introduction

Proper installation of custom HCI clients is very important.



WARNING

Improperly installed clients may cause problems that are often extremely difficult to troubleshoot.

For example, the client may not be able to communicate with HCI servers, or asynchronous callbacks from the server to the client may fail. There are four ways to startup an HCI client.

- Interactive startup (manual)
- Automatic startup at boot time
- Automatic startup at login
- CL Server initiated

Client setup varies with the type of startup.

Regardless of the type of startup, the client author must provide an installation package in order to achieve a proper installation. Because clients can be installed on nodes that do not have the same environment as the node used to develop clients, it may be necessary to include various DLLs required by the client. Also, it may be necessary to register these DLLs. The author of the client installation package must follow standard Microsoft-compliant installation procedures. This includes, but is not limited to

- Logo compliance
- Setting up the registry



ATTENTION

Honeywell uses a product called InstallShield to perform all software installations for TPS and has InstallShield scripts, which facilitate easy and correct installations that follow Honeywell guidelines. Therefore, it is recommended that InstallShield be used to produce custom HCI client installation packages.

TPS installation requirements

Before installing a custom HCI client, you must install the HCI Runtime package. If your HCI Client uses GetComponentInfo, you must also install the System Management Runtime package.

In addition, you must install the appropriate server's client only package for any remote server to which the client connects. This is to ensure that all proxy/stub DLLs, and server registration is accessible to the client.

Security

The custom HCI client installation package must set the appropriate permissions to access the server and the node on which the server runs. If the client and server are in different Windows 2000/2003 domains, then the two Windows 2000/2003 domains must have a two-way trust relationship. For more information, please refer to the following Honeywell document.



REFERENCE - INTERNAL

For more information on the trust relationships, see the section called *Two-way trust relationships* in the *TPS System Implementation Guide*

Startup

The following subsections define the startup methods.

Interactive Startup

This client startup method requires an engineer or operator to log on to the application node and double-click the appropriate client executable, or run the client from the command line (the Run command). In this case, the client will be executed as the user that is currently logged onto the node. This is the easiest way to start a client and is usually appropriate for clients that interact with an operator or engineer.

Automatic Startup at Login

Another easy way to startup a client is to add a shortcut to a specific startup folder. Shortcuts, which are predefined Run commands, can be created to automatically start a custom HCI client. Each node has several different startup folders, which reside in the following locations:

```
\WINNT\Profiles\Default User\Start Menu\Programs\Startup  
\WINNT\Profiles\specific_username\Start Menu\Programs\Startup
```

2 Installation Verification

2.4 Custom Client Installation

If a shortcut exists for the custom HCI client in the appropriate startup folder, then the client will be automatically started when the appropriate user logs onto the node. Clients that are started automatically must account for the possibility that the server is not yet running. This applies to out-of-process servers only. If the server is an inproc server, then there is not a problem. Custom clients should not start the server if the server is not already running, because the server would shutdown when the last client disconnects rather than be shutdown by the TPS system. Having the proper “launch” permissions set for the server would alleviate this problem. HCI servers are designed to do startup and shutdown by the Honeywell System Management Display or the Component Administration Service (CAS).

The login to the application node can also be automated so that an operator is not required to interact with the application node, other than ensure that power is available.

Automatic Startup at Boot Time

If a client must be active when the node is powered on, even if no user is logged onto the node, then the custom HCI client must be written as a service. A service is different from an executable, in that a service can be configured to start at boot time and no user need be logged in for the service to run. Services may also be configured to start manually. Service configuration is performed from the Control Panel, using the Services icon.

To create a custom HCI client as a Service, several additional implementation steps are required. These steps include:

Step	Action
1	Implement a main function that is the entry point of the code. The main function is responsible for parsing any command-line arguments, and installing, starting, and removing the server, and so forth.
2	Implement a function that provides the entry point for the actual service code. Pass the address of this function to the service manager when the service is first started.
3	Implement a function that processes command messages from the service manager.
4	Register your client as a service.

For further details on how to create a Service, please refer to the Microsoft documentation relating to “Creating a Simple Windows NT Service in C++.” After the service has been installed, the “Log On As” account needs to be set to the appropriate user.

To set the “Log On As” account, go to Control Panel/Services/Service Name/Startup. Clients that are started automatically must account for the possibility that the server is not yet running. Custom clients should not start the server if the server is not already running, because the server would shutdown when the last client disconnects rather than be shutdown by the TPS system. Refer to problems referenced above in the section “Automatic Startup at Login.”

CL Server Initiated

Custom HCI clients can also be started with the CL Server. For more information on how to initiate a client with the CL Server, refer to the following documents:



REFERENCE - INTERNAL

[For more information, refer to the *CL Server User's Guide*](#)

Installation package guidelines

Once the mechanism of custom HCI client startup has been determined, the installation packaging can be produced. Honeywell recommends that the InstallShield tool be used for creating installation packages.

2 Installation Verification

2.5 Verifying Installation and Configuration on a TPS Node

2.5 Verifying Installation and Configuration on a TPS Node

Monitor TPS Domains

Once the TPS system is installed and configured, launch the Honeywell System Management Display to monitor your TPS domains. There are two ways to do this.

Start → Programs → Administrative Tools → <ConsoleName>.msc

-or-

Start → Run. At the Run command, type in **mmc** and click **OK**.

Start/Stop the HCI Components

When an HCI component is started it will transition to the idle state. When stopped, it will transition back to the Configured state. If the TPS domain consists of multiple nodes, you can test the remote launch capability by Starting/Stopping an HCI component configured on a different node.

3. Key Concepts

3.1 Honeywell Communications Interface (HCI)

Communications Infrastructure overview

The primary purpose of the communications infrastructure is to support interoperability among TPS clients and servers. Microsoft's Distributed COM provides the basis for distributed object communication and object management. DCOM provides the underlying transport mechanisms that are used to meet the inter-object communication requirements of the TPS system. This includes such items as transparent distributed object communications, server activation security, optional call (method) security, concurrency support, and release independence through immutable interfaces. DCOM is the communication mechanism used between TPS clients and TPS servers.

The overall data access approach for TPS system is compliant with OLE for Process Control (OPC 2.0.5 specification).

3.2 HCI Client Access

Overview

HCI supports both custom and automation interfaces for OPC. The HCI interfaces and remoting will not work directly for VB automation. Therefore, HCI provides a wrapper class that is used around the HCI interfaces and for remoting. This class is called VBHelper, which is utilized by VB clients. HCI interfaces are immutable and will be supported for two releases following their obsolescence.

Create Instance

The illustration that follows, entitled "HCI OutOfProc OPC Server (EXE)," shows the relationship of the client to HCI components. In order to create an instance, the client must either know the ProgID or the HCI component name (also referred to as HciComponentName in TPS Naming Conventions).

Under the ProgID subkey in the registry is the unique CLSID for the class of HCI component. A client may use the CLSIDFromProgID function to obtain the unique CLSID.

An HCI component name is configured in the registry under the HKEY_LOCAL_MACHINE\SOFTWARE\Honeywell\MyTpsDomain\HciComponents. If the ProgID is not known, an HCI client can call IHciClientUtil::GetComponentInfo to determine the ProgID, CLSID, and hostname of the HCI Server component.

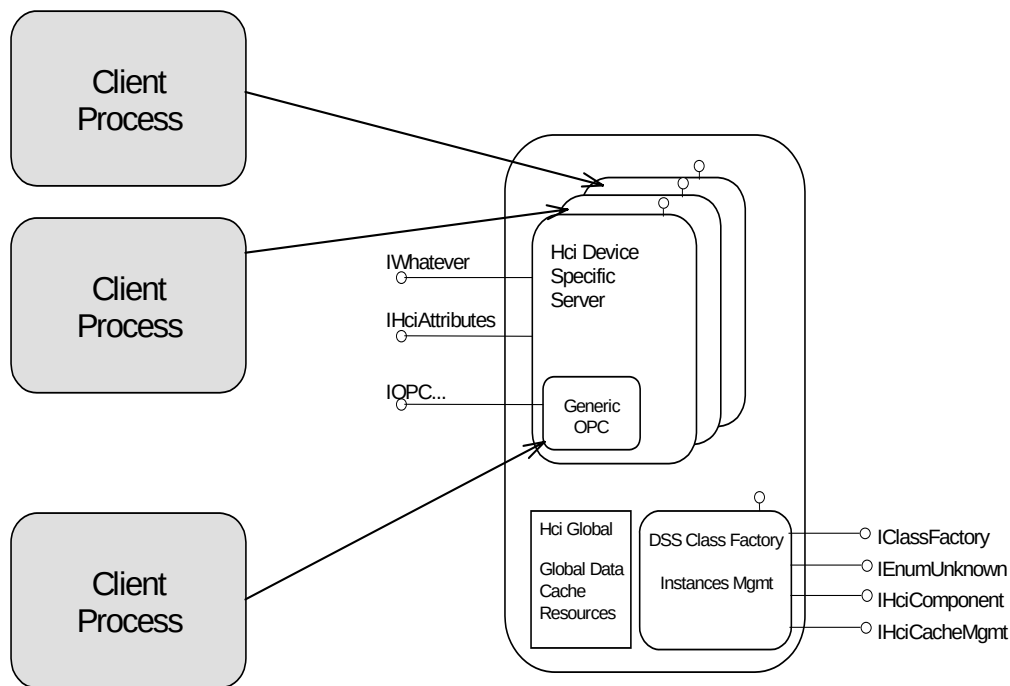
3 Key Concepts

3.2 HCI Client Access

To connect to an HCI component, use CoCreateInstanceEx routine that supports remote instantiation. CoCreateInstanceEx requires the COSERVERINFO parameter to be null if the server is on the local host. Use the IHciClientUtil::IsLocalHost to make this determination. Some HCI components restrict access through the use of runtime licensing, evidenced by a CoCreateInstanceEx return value of CLASS_E_NOTLICENSED. In these cases, the instantiating client must obtain an IClassFactory2 interface pointer using CoGetClassObject followed by a call to the CreateInstanceLic method on this interface. Refer to Microsoft documentation for complete documentation on the IClassFactory2 interface and associated methods.

Clients can Query Interface on the IUnknown pointer returned from CoCreateInstanceEx for outer (IWhatever) or inner (IOPC...) interfaces.

Figure 3 HCI OutOfProc OPC Server (EXE)



Conformance with OPC specification

Data Types

The data type support in OPC is defined for those data types that the Win32 variant supports. HCI does not prevent the usage of any of the VARIANT types. However, you should consult the specific HCI Server guide (for example, *TPN Server User Guide*) for a complete list of VARIANT types supported by the particular HCI Server.

In accordance with OPC, time stamps are in the form of a FILETIME, which are always in UTC.

HCI passes the requested data type as one of the above types to the HCI Server. It is the responsibility of that HCI Server to determine the actual data owner type.

Raw Status

The upper 8 bits of the OPC quality word are available for usage by an HCI Server.



REFERENCE - INTERNAL

Consult the appropriate HCI Server guide for details. An example would be the TPN Server User's Guide.

Special Values

Floating point values such as IEEE POSINF, NEGINF and NaN are allowed as real values (VT_R4, VT_R8). If such a value is returned, it is required that an HCI Server set the quality flags to OPC_QUALITY_BAD.

Asynchronous

The Honeywell Communications Infrastructure supports asynchronous calls for IO for both OPC 1.0A and 2.05 compliant clients.

OPC 1.0A

In Windows 2000/XP/2003, DCOM supports mainly synchronous method invocations. One way to perform asynchronous calls is to use the standard COM `IAdviseSink` interface. To establish an asynchronous entry point for the server to notify the client, the client application must provide an `IAdviseSink` interface pointer to the component object (server) for use in event-driven updates. The OLE for Process Control (OPC) specification relies on the use of `IAdviseSink` in its definition of the `IOPCAsyncIO` interface. In this case, the client must also advise the component object of its

3 Key Concepts

3.2 HCI Client Access

`IAdviseSink` interface, and then invoke a method (`Read`, for example) to force a call to `IAdviseSink::OnDataChange`.

In addition to the asynchronous methods available from the `IOPCAsyncIO` interface, a server ‘message pump’ can be established, which asynchronously notifies the client whenever a predetermined set of data changes. The client sets up the asynchronous entry point in the same manner as described for other asynchronous operations. If this relationship is established, the OPC Group and Items are active, then if the data changes more than the percent deadband, the client will be updated on its periodic update rate.

HCI provides a utility class `HciSink` to simplify the client’s use of `IAdviseSink` (see subsection “Asynchronous Utility Classes” below under section “HCI Utilities”).

OPC 2.0

OPC 2.0 uses connection points with `IOPCDataCallback` as the asynchronous IO mechanism, which replaces the `IDataObject/IAdviseSink` mechanism described above. Connection points are an alternative COM mechanism for registering and revoking callback interfaces with a server. In order to use connection points, the client must create an object that supports the `IOPCDataCallback`, which serves as the “advise” implementation. The `OPCGroup` object within an HCI server exposes `IConnectionPointContainer` through `QueryInterface`. This enables a client to associate its “advise” implementation with an HCI Server.

In addition to the asynchronous methods available from the `IOPCAsyncIO2` interface, a server ‘message pump’ can be established, which asynchronously notifies the client whenever a predetermined set of data changes. The client sets up the asynchronous entry point using Connection Points rather than `IDataObject`. If this relationship is established, the OPC Group and Items are active, then if the data changes more than the percent deadband, the client’s `IOPCDataCallback::OnDataChange` will be updated on its periodic update rate.

COM Client security considerations

Several specific aspects of client security are addressed in this section. They are:

- The initialization of the COM security environment, which is applicable to ALL client applications, and
- The use of OPC Security, which may or may not be applicable to a specific client application.
- Text Credentials
- Impersonation-based OPC Security
- Security Checklist for COM Clients

COM Security Initialization

COM clients should always call `CoInitializeSecurity` right after the first call to `CoInitialize` within the client process. If the client does not use this API, it will run with the machine default security settings. This may result in call failure (if impersonate level is too low), poor performance (if authentication level is too high), or other unpredictable behavior.

The `CoInitializeSecurity` call should include:

- Authentication level: as needed by the client. This will possibly be negotiated upward by the server.
- Impersonation level: that the client will allow. Note that servers may refuse calls if this level is too low for the server to accomplish its task.
- Other settings may be appropriate to specific clients.

Server and client developers may each have ideas about the required authentication level appropriate to the connection between them. For this reason, Windows negotiates the authentication level for the connection to the higher of the levels specified by the client and the server. Developers should be cautious about setting the authentication level too high, as it can have significant performance impact. HCI requires a minimum authentication level of `RPC_C_AUTHN_LEVEL_CONNECT`. Some applications that are concerned about data integrity or privacy may choose higher authentication levels, but will incur considerably more overhead. Clients using `RPC_C_AUTHN_LEVEL_NONE` will normally be refused connection to HCI servers.

Clients may also want to be careful about setting impersonation level too high, since a server called with impersonate level of `RPC_C_IMP_LEVEL_IMPERSONATE`, or above, may do anything on the server machine that the client user is allowed to do, which could be dangerous. Typical HCI servers will refuse calls that arrive with `RPC_C_IMP_LEVEL_ANONYMOUS`. In some cases, client developers may want to allow the interactive user to influence the impersonation level used, if there is reason for a user to require greater data integrity or privacy.

It is rather easy and straightforward to upgrade authentication for significant exchanges between the client and server. As an example, if the client wanted to send a password to the server encrypted, but most other information did not require that level of security, the client could do the following:

3 Key Concepts

3.2 HCI Client Access

Step	Action
1	Initialize security with authentication at Connect level.
2	Connect to server and carry on initial dialog.
3	Upgrade security for the single exchange that transfers the password by performing these sub steps: a) Create a copy of the interface pointer using CoCopyProxy(). b) Set the security blanket of the copied interface pointer only, using CoSetProxyBlanket(), setting authentication to Packet with privacy level (RPC_C_AUTHN_LEVEL_PKT_PRIVACY). c) Make the call passing the password. d) Release() and destroy the copied interface pointer.
4	Carry on the server interaction using the original interface pointer(s) at the original authentication level.

OPC Security Considerations

The OPC Security Custom Interface Specification defines an optional custom interface that servers may implement to give client applications more flexibility for access security. A sample client that illustrates the usage of OPC Security is included in the HCI Client Development toolkit (CopClient.exe).

The OPC Security specification defines a mechanism that clients can use to notify the server that the security of future calls should be based on a user that is different from the current Windows login.

OPC Security defines two interfaces. IOPCSecurityPrivate will be implemented by servers that use private security mechanisms, usually based on a proprietary list of users and passwords. This interface is not supported by HCI servers. The other interface (IOPCSecurityNT) is implemented by servers that use Windows user IDs and passwords as the basis for security. The foundation support for this interface is supplied by HCI, making support by HCI servers relatively easy.

This section describes the impact on a client application to support using the IOPCSecurityNT interface. As noted above, the CopClient sample in the HCI Client Development toolkit illustrates various security options available to client developers.

The core of the OPC NT-based security is the `IOPCSecurityNT::ChangeUser()` call. When a client makes that call to an HCI server that supports OPC security, the COM identity used for the call is captured by the HCI server, and used as the basis for all succeeding calls, until another `ChangeUser` call is made, or the connection is broken. So the primary job of the client is to correctly establish the COM identity for the `ChangeUser` call, based on whatever mechanism the client chooses.

COM implements the `CoSetProxyBlanket` call which supports two mechanisms for setting the call identity for a COM call. One way is to supply textual credentials (domain, user, and password) to the `CoSetProxyBlanket` call. The other requires the client to impersonate the new user, then make the `CoSetProxyBlanket` call with the ***cloaking*** flag set correctly. There are several issues the client developer must address when deciding which mechanism to use.

Text Credentials

Before deciding to use text credentials, the client developer must understand that there is one essential limitation imposed by COM: if the client and the server are on the same machine, text credentials are ignored. They are not validated. They have no impact whatever on the COM connection to the server. No errors are returned that indicate the credentials are not being used.

The steps a client must implement to make use of OPC Security are as follows.

Step	Action
1	QueryInterface on another server interface for the <code>IOPCSecurityNT</code> interface.
2	Call <code>IsAvailableNT()</code> on this interface to verify that NT security is supported.
3	Call <code>QueryMinImpersonationLevel()</code> on the security interface, and determine if the client is willing to allow that level of impersonation to this server.
4	Obtain text credentials (user id, domain, and password), either interactively from the user, or from another source.
5	Create and initialize a <code>SEC_WINNT_AUTH_IDENTITY</code> structure using the text credentials.
6	Call <code>CoSetProxyBlanket()</code> passing the <code>SEC_WINNT_AUTH_IDENTITY</code> structure and an impersonation level at least as high as the value returned from <code>QueryMinImpersonationLevel()</code> .
7	Call <code>ChangeUser()</code> on the security interface.

3 Key Concepts

3.2 HCI Client Access

Step	Action
8	If this is an HCI server, reset the proxy blanket. NOTE: Other servers may require the proxy blanket to remain.
9	Make calls on other server interfaces that will be based on the new identity.

One example of an implementation of these steps is shown in the CopClient sample in the \ProgramFiles\Honeywell\TPS\Toolkits\Hci section of this document.

Impersonation-based OPC Security

In Windows 2000 and beyond, COM supports a form of the CoSetProxyBlanket that uses the impersonation identity of the caller. To do this, the call must specify EOAC_STATIC_CLOAKING or EOAC_DYNAMIC_CLOAKING. Prior to Windows 2000 (or if cloaking is not specified on Windows 2000 and beyond) the impersonation state is ignored by COM.

One of the primary advantages of using cloaking with OPC Security is that it works the same way whether the server is on the same machine as the client or not.

The steps a client must implement to use impersonation-based OPC Security are as follows.

Step	Action
1	QueryInterface on another server interface for the IOPCSecurityNT interface.
2	Call IsAvailableNT() on this interface to verify that NT security is supported.
3	Call QueryMinImpersonationLevel() on the security interface, and determine if the client is willing to allow that level of impersonation to this server.
4	Impersonate the new user. (For more details, see the comments that follow this procedure.)
5	Call CoSetProxyBlanket() passing the appropriate cloaking argument and an impersonation level at least as high as the value returned from QueryMinImpersonationLevel().
6	Call ChangeUser() on the security interface.
7	The proxy blanket can be reset now if this is an HCI server – other servers may require the proxy blanket to remain.
8	Make calls on other server interfaces that will be based on the new identity.

Step 4 in the prior procedure requires the client to impersonate the desired user. This is not necessarily easy for a client application running on behalf of an interactive user. Here are three possible methods you could use so the client can impersonate the desired user.

- If this client application is also a server, Windows make it easy to impersonate the RPC, named pipe, or COM client responsible for this call. Normally, this is not the case.
- Another way is to make the Win32 call LogonUser(), passing text user, domain, and password. However, this is not generally a reasonable solution for a client

3 Key Concepts

3.2 HCI Client Access

application, as the user that owns the process must be granted the right to “Act as part of the operating system”, which is uncommon, and a definite security risk.

- A third mechanism is illustrated in the CopClient sample in the \ProgramFiles\Honeywell\TPS\Toolkits\Hci section of this document. This sample connects to the Honeywell Signon Manager, and is notified each time a new user signs on to the system using the Signon Manager. At the time of this notification, the client is able to save a handle to a “token” which defines the currently signed-on user. It is this handle which is passed to the Win32 function ImpersonateLoggedOnUser().

Security Checklist for COM Clients

Ask the following questions when determining security for COM clients.

Checklist Question	✓
Does client call CoInitializeSecurity() with security levels appropriate to client's server access needs?	
Does client increase security on server interactions that transfer sensitive information?	
If supporting OPC Security, are the steps listed above implemented as appropriate?	

Method Access security

An HCI server component will deny access to clients who do not have adequate permissions on the capability associated with the component method. Not all methods of a given HCI component are secured. If the client does not have sufficient access rights, the method immediately returns E_ACCESSDENIED.

An HCI server component has the following base methods, configurable to be secured:

- Shutdown
- OPC Read
- OPC Write

The HCI server component also may have methods in addition to the OPC read and write methods, which are secured.



REFERENCE - INTERNAL

Consult the appropriate HCI Server guide(s) for details. An example would be the TPN Server User's Guide.

Server Attributes

HCI provides an HCI-specific method, which allows a client to specify "known properties" of the server. These "known properties" are referred to as Server Attributes; one of which is "priority." Such attributes may be "TPS-wide" in that by convention, all HCI Servers support them, or they may be server-specific (supported by only one or a few servers). An example of such a server-specific attribute is the "access level" attribute supported by the HCI TPN Server.

The interface, IHciAttributes, on an HCI Server supports methods to get and set server attributes at any time. Clients using C++ use QueryInterface to obtain a pointer to this interface, while automation clients must access this object through the IDispatch interface. You can think of IHciAttributes as a "required" IWhatever for an HCI server. HCI provides the framework into which the HCI server author provides the "real" server attribute support.

3 Key Concepts

3.2 HCI Client Access

Because each HCI server provides the real server attribute support, the client developer is referred to the specific HCI server guide for the following:

- Definition and/or configuration of server attribute values
- Names of attributes supported. These names are required as input parameters to get and set methods.
- Associated security levels defined per attribute supported



REFERENCE - INTERNAL

Consult the appropriate HCI Server guide(s) for details. An example would be the TPN Server User's Guide.

Attribute settings are in effect for only the server instance that is connected to that particular client.

A specified server attribute value remains in effect until the client requests to set a new value for the same server attribute. If the client does not explicitly request to set attribute values, all requests are executed at the default attribute values established by an HCI server.

The ability to set server attributes can be secured at the attribute level. An HCI server establishes this security.

Priority

"Priority" is considered a TPS-wide attribute. That is, if supported by a server, it is consistent with all other HCI servers that support "Priority." Given the general server attribute mechanism described previously, the specified priority level is passed to an HCI server by `IHciAttributes::SetAttributes`, and is used to prioritize subsequent requests from this client relative to concurrent requests from other clients. By convention, ten priority levels are supported. If the underlying device does not support a priority mechanism, the HCI server will ignore the HCI priority settings.

3.3 HCI Utilities

Overview

This static class library contains a set of utility classes to simplify client application development. These classes attempt to maximize reuse through building blocks. The functionality provided by these classes can be generalized into the following areas that are outlined below:

- Class to assist in the setup of event-driven or asynchronous clients
- Class to manipulate data returned in streams.

Asynchronous Utility classes

Client applications that wish to use the asynchronous mechanisms provided by HCI and DCOM may opt to utilize the `HciSink` class to support the callbacks received from component objects. HCI encapsulates the `IAdviseSink` interface with a callback oriented API (`HciSink` class).

Instead of each client implementing its own object that supports `IAdviseSink`, this utility class can be used, and the client need only implement a callback function.

3 Key Concepts

3.3 HCI Utilities

Data Manipulation Utility classes

The OPC specification defines the data structures and data packaging for HCI Server Component objects in the TPS system. All asynchronous data will be returned in the form of a stream. This specifically applies to asynchronous IO and Data Change. HCI provides utility classes to extract the data from the stream into more accessible data types. For example, each data item in the stream has an integer quality, a timestamp, and a variant value. The HCI utility class for streams encapsulates all stream interfaces that are needed to copy these values from the stream into local variables in the client application.

4. Developing an HCI Client Application

4.1 Overview

The HCI client application may be developed using any of the software development environments supported on Windows 2000/XP/2003. A prime example of such an environment is Microsoft's Visual Studio, Enterprise Edition.

4.2 Naming Conventions and Standards

TPS naming structure

Data fetched and stored from HCI Components through HCI is generally modeled as a TPS object, with the TPS extension of the data point. The form of the name for such an access is:

//Domain_name/Server_name/Server_specific_name

Where:

Domain_name = The name of the TPS Domain – the name space of the set of HCI component names being used. If omitted, the default is the local domain.

Server_name = The name of the HCI Component containing the object being accessed.

Server_specific_name = The TPS object name within the HCI component. It may be a multi-point name, indicating structure of component objects, such as "F100.PV". TPN parameters such as PV, SP, Mode, and so forth are modeled as component objects, not properties. TPS object names are not limited to two parts (one level of containment) as with TPN data points. A Server_specific_name may also be an entire or partial TPS name (That is //MyTPSDomain1/TPNServer1/A100.PV or /TPNServer1/A100.PV).

The OPC ItemID or Server-specific name, which is passed to the HCI component, must conform to the HCI server specification. Each HCI Component is responsible for handling the Domain name and Server name. Some HCI Components may ignore the Domain name and/or Server name. Please refer to the appropriate HCI server documentation to validate how the TPS Naming structure is handled.

4.3 Threading Model

An HCI/OPC client can choose to be either apartment threaded or free threaded. Clients that choose to be apartment threaded are responsible for marshalling/unmarshalling any interface pointers that are passed between threads. Free threaded clients are able to pass interface pointers between threads at will. HCI servers communicate with clients implemented in either threading model.



REFERENCE - EXTERNAL

See Microsoft documentation for a complete description of threading options.

4.4 Using the Custom Interface vs. the Automation Interface

Which interface should the client application use

In general, client programs created using scripting languages will use the automation interface. Client programs created in C++ will find it easiest to use the custom interface for maximum performance.

In the following API reference sections, the interface descriptions are organized by object.

4.5 Custom Application Scenario

Using Custom Interface

The following scenario is for a typical client application using the custom interface. The client interface will likely use the following sequence of method invocations to access data through HCI:



ATTENTION

The intent of this sample scenario is to provide a general information guideline only and is not intended to be a comprehensive example of all interface activities.

HCI Interface

- Obtain the information needed to connect to the desired server using `IHciClientUtil::GetComponentInfo`.



ATTENTION

IHciClientUtil is the COM version of the HCI class that is currently available in the static library, `hciutil.lib`. HCI is providing some client interfaces as COM objects in order to provide Release Independence from previous HCI releases. **IHciClientUtil** is an interface of **HciClientUtil** COM object. This COM object is provided within `clientutil.dll`. The names and functionality of the methods are the same as those provided by **Hci** class. However, the method signatures are modified in order to be COM compliant. Use of **IHciClientUtil** methods requires connection to the inproc server **HciClientUtil**.



ATTENTION

If you are using an OPC server that is unaware of HCI, then you must either obtain this information through other sources or "configure" the 3rd party OPC server with the TPS Domain Configuration utility. In this context, "configure" means to associate a TPS name for the 3rd party OPC server that has been installed on a TPS node.

- Connect to the server by calling the DCOM function `CoCreateInstanceEx`.
- If specific security settings are required, call `CoInitializeSecurity`.

OPC Interface

- Define one or more group(s) of data items to be accessed using `IOPCServer::AddGroup`.
- Add data items to the groups using `IOPCItemGrp::AddItems`.
- Read and write data items using `IOPCSyncIO::Read`, `IOPCSyncIO::Write`, `IOPCAsyncIO::Read`, or `IOPCAsyncIO::Write`.



ATTENTION

If you're using asynchronous I/O, you may want to use methods supplied by the **HciSink** Class that deal with Callbacks.

DCOM Interface

- When finished, the client terminates its connection to the server by invoking the DCOM function `Release()` on the pointer that was returned from `CoCreateInstanceEx`. If you have connected to `HciClientUtil` COM object, you must also invoke the DCOM function `Release()` on the pointer that was returned from `CoCreateInstanceEx`.

4.6 Automation Application Scenario (OPC 1.0A)

Using Automation Interface

The following scenario is for a typical Visual Basic client application using the automation interface. The client interface will likely use the following sequence of method invocations to access data through HCI:



ATTENTION

The intent of this sample scenario is to provide a general information guideline only and is not intended to be a comprehensive example of all interface activities.

HCI Interface

- Initialize the `CHciVBHelper` Class. For VB 5.0 and above the most efficient way to do this is to add a project reference to “Program Files\TPS\Hci\clientutil.dll”, then declare a variable of type `CHciVBHelper` (“Dim helper as new `CHciVBHelper`”)
- Use the `CreateHciObject` method to connect to the desired HCI server.



ATTENTION

If you're using a remote OPC server that is unaware of HCI `CHciVBHelper`'s `CreateRemoteObject` method should be used. VB 6 also allows `CreateObject` to accept a remote machine as an optional second parameter.

OPC Interface

- Define one or more groups of data items to be accessed using the `IOPCServerDisp` object's `AddGroup` method.
- Add data items to each group using `IOPCItemMgtDisp` `AddItems` method.

- Read and write data items using IOPCSyncIODisp OPCRead, and OPCWrite methods, or IOPCAsyncIODisp OPCRead, and OPCWrite methods.



ATTENTION

If you're using asynchronous I/O, then you may want to use the IOPCAsyncIODisp AddCallbackReference method to specify a Callback object.

Server Disconnect

- When finished, the client releases its connection to the server.

4.7 Automation Application Scenario (OPC 2.0)

Using Automation Interface

The following scenario is for a typical Visual Basic client application using the OPC 2.0 automation interface. The client interface will likely use the following sequence of method invocations to access data through HCI:



ATTENTION

The intent of this sample scenario is to provide a general information guideline only and is not intended to be a comprehensive example of all interface activities.

HCI Interface

- Initialize the CHciVBHelper Class. For VB 5.0 and above the most efficient way to do this is to add a project reference to "Program Files\TPS\Hci\clientutil.dll", then declare a variable of type CHciVBHelper ("Dim helper as new CHciVBHelper")
- Use the VB helper to obtain the ProgID - helper.GetComponentInfo "component_name", clsid, progid, host
- Use the OPCServer::Connect method to connect to the desired HCI server.

```
Set server = New OPCServer  
server.Connect (progid)
```

OPC Interface

- Define one or more groups of data items to be accessed using the `OPCGroups` object's `Add` method.
- Add data items to each group using `OPCItems` `AddItems` method.
- Read and write data items using `OPCGroup` `SyncRead`, and `SyncWrite` methods, or `AsyncRead` `AsyncWrite` methods.



ATTENTION

If you're using asynchronous I/O, then use the `IOPCGroupEvent` `AsyncReadComplete` and `AsyncWriteComplete`.

Server Disconnect

- When finished, the client releases its connection to the server by calling the `OPCServer::Disconnect` method.

```
server.Disconnect (progid)
```

5. HciTimeout Object

5.1 Overview

Introduction

This section describes the HciTimeout object that is available to HCI clients interfacing with HCI/OPC servers.

5.2 HciTimeout

Functional description

The HciTimeout object is available to HCI clients interfacing with HCI/OPC or OPC servers. The HciTimeout object provides the ability to invoke each supported OPC interface method, as well as a set of HCI interface methods with a timeout. Therefore, control is returned to the client if the request has not completed within a specified amount of time.

For the IOPCAsyncIO or IOPCAsyncIO2 methods, only the synchronous part of the call is actually timed. The callback containing the results is not timed.

In most cases, using the HciTimeout object does not make sense if the server is an inproc DLL, which is loaded in the client. This would only cause unnecessary overhead.

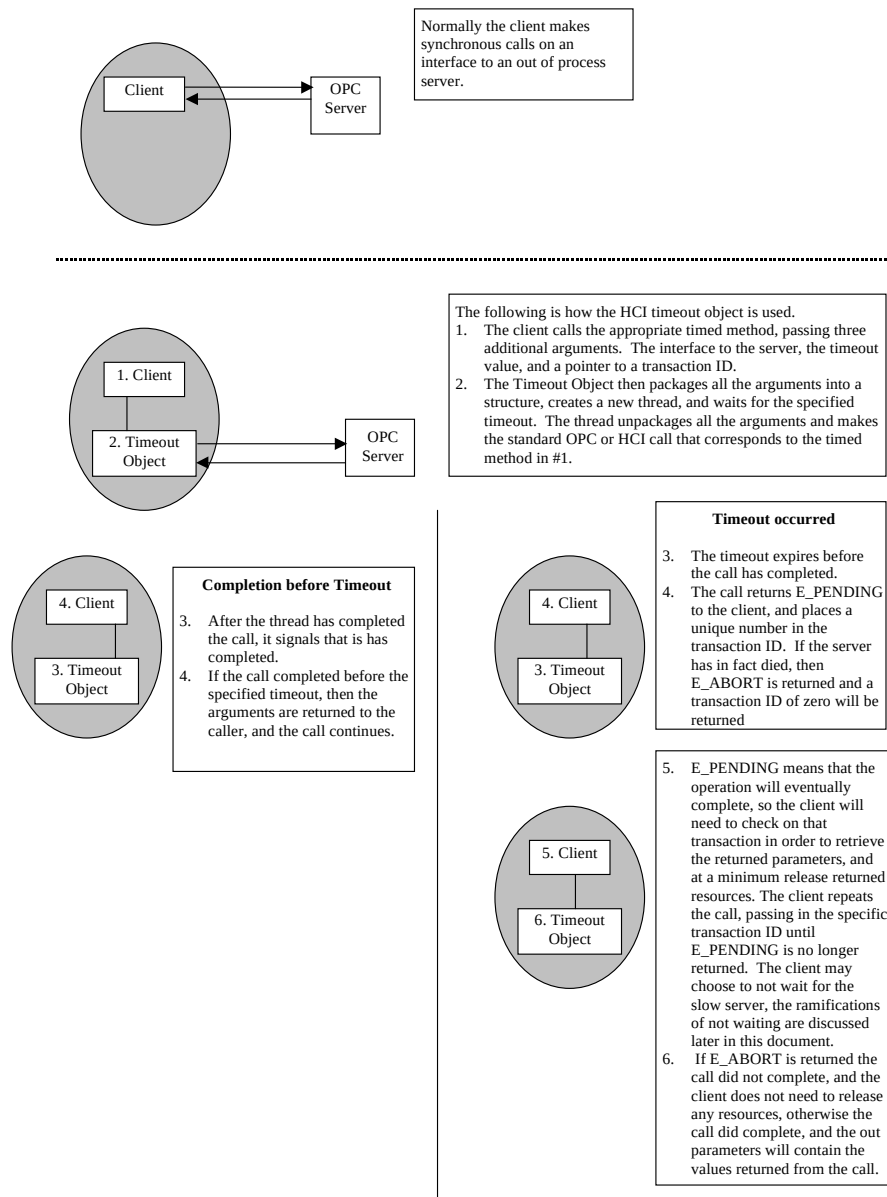
The following figure illustrates the internal functionality of the HciTimeout object as compared to a request with no timeout.

This class contains static methods useful to HCI client and application developers. These methods allow an HCI client to easily obtain the required information necessary to interface with an HCI Component object.

5 HciTimeout Object

5.2 HciTimeout

Figure 4 Internal Functionality of the HciTimeout Object



In general, there are three possible results from a request through the HciTimeout object:

1. The operation completes within the specified timeout period. The results are returned to the client in the original timed request.
2. The operation returns immediately. This indicates that the server is not responding and may be dead.
3. The operation returns after the specified time expires. This indicates that the server is slow. In this case, there are three possibilities:
 - The server is still processing the request. The operation will eventually complete with the results available for the client.
 - The server is hung or dies while processing the request. The client does not receive any results, although the timeout status is returned to the client.
 - The client decides to continue to wait on the request. The client becomes responsible for the future state of the server as well as cleaning up any memory or resources that are returned by the server.

Public Methods summary

The following interfaces and methods are available from the HciTimeout object:

Table 6 HciTimeout Interfaces and Methods

Interface/Method	Interface/Method
IHciTimeout CheckStatus	IOPCGroupStateMgtTimed GetState SetName SetState CloneGroup
IOPCCommonTimed SetLocaleID GetErrorString GetLocaleID SetClientName QueryAvailableLocaleIDs	IOPCSyncIOTimed Read Write

5 HciTimeout Object

5.2 HciTimeout

Interface/Method	Interface/Method
IOPCServerTimed AddGroup GetStatus GetErrorString RemoveGroup GetGroupByName CreateGroupEnumerator	IOPCAsyncIOTimed Read Refresh Write Cancel
IOPCAsyncIO2Timed Read Refresh2 Write SetEnable Cancel2 GetEnable	IOPCItemMgtTimed AddItems SetActiveState ValidateItems SetClientHandles RemoveItems SetDatatypes CreateEnumerator
IEnumOPCItemAttributesTimed Next Reset Skip Clone	IOPCItemPropertiesTimed QueryAvailableProperties GetItemProperties LookupItemIDs
IHciTimeoutDisp Status	IOPCServerDispTimed Count VendorInfo StartTime AddGroup CurrentTime GetErrorString LastUpdateTime GetGroupByName MajorVersion RemoveGroup MinorVersion SaveConfig BuildNumber LoadConfig

Interface/Method	Interface/Method
IOPCServerPublicGroupsDispTimed GetPublicGroupByName RemovePublicGroup	IOPCBrowseServerAddressSpaceDisp Organization ChangeBrowsePosition SetItemIDEnumerator GetItemIDString SetAccessPathEnumerator
IOPCItemMgtDispTimed Count RemoveItems Item SetActiveState AddItems SetClientHandles ValidateItems SetDatatypes	IOPCGroupStateMgtDispTimed ActiveStatus LCID Name CloneGroup UpdateRate TimeBias PercentDeadBand ClientGroupHandle ServerGroupHandle
IOPCSyncIODispTimed OPCRead OPCWrite	IOPCAsyncIODispTimed OPCRead Cancel OPCWrite Refresh AddCallbackReference DropCallbackReference

5 HciTimeout Object

5.2 HciTimeout

Interface/Method	Interface/Method
IOPCItemDispTimed AccessPath Quality AccessRights Timestamp ActiveStatus ReadError Blob WriteError ClientHandle EUType ItemID EUInfo ServerHandle OPCRead Value OPCWrite RequestedDataType	IOPCPublicGroupStateMgtDispTimed State MoveToPublic
IHciAttributesTimed SetAttributes GetAttributes	IHciPulseTimed Configure CancelPulse Start GetInfo
IUnknownTimed QueryInterfaceTimed AddRefTimed ReleaseTimed	IHciComTimed CoCreateInstanceEx
IDataObjectTimed DAdviseTimed DUnadviseTimed	IConnectionPointContainerTimed EnumConnectionPoints FindConnectionPoint
IConnectionPointTimed Advise Unadvise	IEnumConnectionPointsTimed Next Skip/Reset Clone
IEnumConnectionsTimed Next Reset Skip Clone	OPC 2.0 Automation Interfaces

HCI provides a timed version of each method and property as defined in the OPC Data Access Automation Specification, version 2.02, dated February 4, 1999. The HCI timeout version appends “Timed” to the method name and includes three additional parameters, which are inserted prior to those specified for the OPC 2.0 method. These three parameters are: pDisp as Object, lTimeout as Long, lTransID as Long.

For example, the VB signature of the HCI timed version of: IOPCGroup::SyncRead

Would be:

```
IOPCGroupTimed::SyncRead (pDisp as Object, lTimeout as Long,  
lTransID as Long, Source As Integer, NumItems As Long,  
ServerHandles() As Long, ByRef Values() As Variant, ByRef  
Errors() As Long, Optional ByRef Qualities As Variant, Optional  
ByRef TimeStamps As Variant)
```



REFERENCE - INTERNAL

For more information, refer to the *OPC Data Access Specification Reference Manual*. You will find the *OPC Data Access Automation Specification* in that manual.

AnyTimedCustomMethod

```
STDMETHODIMP AnyTimedCustomMethod(AppropriateInterfacePointer *pIntf,  
DWORD dwTimeout,  
DWORD *pdwTransID,  
... original parameters );
```

Description

Use “AnyTimedCustomMethod” (see the previously defined Public Methods Summary) to:

- Make an initial OPC or HCI timed request to a server.
- Obtain the results from a previously timed request that finished after the timeout period.

The HciTimeout object is an ATL COM object, which is loaded as an inproc DLL. Installing either the HCI Runtime or HCI Client Developers Toolkit packages will setup the registry and perform the necessary registrations of the HciTimeout object.

The client uses standard COM methods to access the object. CoCreateInstance() on CLSID_HciTimeout is used to load the DLL into the clients process. Then, to obtain a pointer into the Timeout object, call QueryInterface on a timeout interface such as

5 HciTimeout Object

5.2 HciTimeout

IID_IHciTimeout. The Timeout object supports both OPC and HCI methods as listed in the Public Methods Summary.

Each interface name is appended with the word “Timed.” For example, the OPC interface IOPCSyncIO is implemented in the Timeout object by the interface named IOPCSyncIOTimed. All timed interfaces consist of methods, which have a one-to-one correspondence to the methods of non-timed OPC or HCI interfaces.

Each timed method has three additional parameters, which are inserted before the parameters of the original OPC or HCI signature. The first parameter is an interface pointer to the actual OPC or HCI interface. The interface pointer must be a pointer to the appropriate OPC or HCI interface.

The second parameter is the desired timeout period in milliseconds. This is of type DWORD. The minimum allowed timeout value is 100 milliseconds. If a timeout period of 1-99 is passed, then 100 milliseconds will be used. If the client wants to use the timed interfaces with no timeout, then this parameter is specified as 0. Since the timeout parameter is unsigned, a negative value should not be used because this would actually translate to a very large positive number. This large timeout value will simulate waiting for a very long time period. The client may want to take advantage of this for ease of code maintenance. Use of the HciTimeout object in this manner adds some overhead to an OPC or HCI request.

The third timeout parameter is a transaction ID of type DWORD. This is an inout parameter, which has a value of 0 or 1 at input. If the value is 0, then the call is considered to be an initial invocation. An input value of 1, which is only valid for synchronous Read/Write and GetStatus, means that if the operation times out with a state of slow server, the HciTimeout object does not report the results to the client and the HciTimeout object cleans up the memory of the operation that completed after the timeout period.

The transaction ID parameter is returned to the client. A returned transaction ID value equal to 0 means the operation was successful and the results are in the out parameters. A returned transaction ID value greater than 1 means that the server is still processing the request. This new transaction ID can be used to retrieve operation state and results from the Timeout object. In other words, the returned transaction ID is used to monitor the operation’s state while it is E_PENDING.

The HRESULT return codes include those codes that correspond to HRESULTS defined for the particular OPC or HCI method. Additionally, the HciTimeout object may return several other codes. E_PENDING is returned if the server is slow. E_ABORT is returned if the server is not responding. When a server transitions from slow to completed, the OPC or HCI HRESULT is returned. If the operation does not timeout, then any valid OPC or HCI HRESULT is returned.

If the operation times out because of a slow server, the client is responsible for monitoring the status of the operation. Furthermore, the client is obligated to free any memory that the client would have originally been responsible for freeing. The IHciTimeout interface provides two ways to monitor a slow server:

1. The CheckStatus method is available to monitor an operation that has been deemed slow. This method returns an HRESULT indicating the status of the operation. The status is one of three possible states: slow, dead/hung, or completed. If the operation status changes to dead/hung (E_ABORT), then the client does not need to continue to monitor the operation. The operation is considered complete. If the state continues to be slow, then the client must wait until the state changes to either dead/hung or complete. If the state changes to complete, the client can retrieve the results and continue.
2. Continue to call the original timed method with the returned transaction ID while monitoring the HRESULT. If the HRESULT changes from E_PENDING to a new condition, the operation is complete and the results are in the original out parameters, assuming the operation successfully completed. It is suggested that the client wait for a small amount of time before recalling the original timed method. (See the sample code.)

Once the state of the operation changes from slow (E_PENDING) to a different HRESULT, the client must call the original method to obtain the results. In the original request, the client passed a transaction ID of zero. This transaction ID was assigned a new value by the HciTimeout object and returned to the client in the case where the operation timed out. When retrieving the results from an operation that transitions from slow to complete, the client makes the original call with the returned transaction ID. The HciTimeout object tests the transaction ID to validate that the original request has successfully completed. If successful, the HciTimeout object returns the results in the original out parameters.

5 HciTimeout Object

5.2 HciTimeout

Client Timeout Responsibilities

It is important to understand that when a timeout occurs with a state of slow server (HRESULT of E_PENDING), the client should not proceed unless the client is willing to accept the unknown future state of the server. If the client chooses to continue, it must return to cleanup memory and resources later.

In general, the client should wait until the operation actually times out with a dead/hung server (HRESULT of E_ABORT), or completes. The HCI/OPC and OPC servers are not transaction based, and therefore do not have the ability to rollback requests. In most cases, slow servers will complete the request. This means that the state of the server may have changed depending on the operation performed.

In addition, there may be out parameters that the client is responsible for freeing. If the client continued to process after the receipt of a slow server state, memory leaks and/or server state will become indeterminate. For example, if the client performed an IOPCServerTimed::RemoveGroup(...) and is determined to be slow.

The client is not sure if the RemoveGroup is going to happen or not, unless it monitors the operation state. If the operation successfully completes, the group is no longer valid. Awareness of this state of the server is important information for the client.

The client may ignore a slow server state status in the following case: the request is synchronous Read, synchronous Write, or GetStatus *AND* the initial input transaction ID must be set to 1. In this case only, the HciTimeout object automatically cleans up memory upon completion. The client must be aware that the operation most likely will still occur; however, the client is not required to wait for the completion of the operation. The timeout object performs the memory cleanup and discards the results. In this case, if the server times out with a slow state, the results are not available to the client; so the client does not know if the actual operation was performed.

Parameter Specifications

Table 7 AnyTimedCustomMethod Parameter Specifications

Parameter	Direction	Description
pIntfPtr	in	Interface pointer to appropriate interface. For example, if performing a synchronous Timed Read, the IOPCSyncIO interface pointer is required. This must be obtained through a QueryInterface() on the OPC server.
dwTimeout	in	Number of milliseconds to wait for operation to complete before returning control to the client. 100 milliseconds is the minimum timeout period. If 1-99 is passed, 100 milliseconds will be used. If 0 is passed, the operation will not timeout; it will wait until complete, just like an untimed OPC or HCI invocation. This is less efficient than an untimed OPC or HCI call. Since the timeout parameter is unsigned, a negative value should not be used because this would actually translate to a very large positive number. This large timeout value will simulate waiting for a very long time period.
pdwTransID	inout	On the first call to a timed method, the transaction ID should have an input value of 0. For Synchronous OPC Read, Synchronous OPC Write and GetStatus the initial value can be 1. This indicates that if the server is not finished within the specified timeout, then the client can proceed because the HciTimeout object cleans up the results and any outstanding memory from this invocation. If the server is slow and does not complete in the specified time period, control is returned to the client with a transaction ID generated by the Timeout object. This returned transaction ID is to be used to get the state of the operation and the results once the server finishes with the operation. If the invocation completed before the timeout period expired, the returned transaction ID is 0.
All other method parameters		See the OPC Specification Reference Manual.

HRESULT Return Codes

Table 8 AnyTimedCustomMethod HRESULT Return Codes

Return Code	Description
S_OK	If this HRESULT is returned after an E_PENDING HRESULT, that indicates that the slow server has completed and the results are available for the client. If this is the initial HRESULT, then function was successful.
E_PENDING	The server is slow. The client must wait for the server to complete its operation and retrieve the results once the HRESULTS transitions to S_OK. If the operation is a Sync Read or Sync Write and the original input transaction ID was 1, then the client can continue because the Timeout object will remove the results and cleanup any memory associated with the operation.
E_ABORT	Server is not responding (dead/hung). The client does not receive any results; should continue with other operations.
E_POINTER	The interface pointer or transaction ID pointer is invalid.
E_UNEXPECTED	An HciTimeout object error was detected. Errors include: • Bad transaction ID • Returned values no longer available • Maximum number of outstanding threadsexceeded
E_OUTOFMEMORY	Client is out of memory.
Other method specific HRESULTS	See OPC specification 2.0 and the HCI Client Developers Guide for a complete list of possible return codes.

Comments

It is possible for the client to implement its own Timeout methods for operations that may not be supported by the HciTimeout object. HCI has provided a sample template that can be used as a framework for a client implemented timeout method. This template is located at the end of this section.

The client has the ability to monitor the server, or test to see if the server is alive. To do this, the client should use the OPCServerTimed::GetStatus() method with a timeout value of the client's choice. IOCPServerTimed::GetStatus() would basically be performing a "ping" functionality from client to server.

Sample code Custom

Below is a sampling of the client-side use of the Hci Timeout object.

```

IHciTimeout *pTimeout = NULL;
LPWSTR pNode = L"local";
if (wcscmp(pNode, L"local") == 0)
    pNode = NULL;
COSERVERINFO *pcsi = NULL;
COSERVERINFO csi;
memset(&csi, 0, sizeof(COSERVERINFO));
csi.pwszName = pNode;
pcsi = &csi;
DWORD dwCtx = CLSCTX_INPROC_SERVER;
MULTI_QI mqAll[1];
mqAll[0].pIID = &IID_IUnknown;
mqAll[0].pItf = NULL;
mqAll[0].hr = S_OK;

hr = CoCreateInstanceEx (CLSID_HciTimeout, // classid
                        NULL, // aggregation
                        dwCtx, // server Context
                        pcsi, // machine specific name
                        1, // number of interfaces
                        mqAll); // array of interfaces

hr = pUnk->QueryInterface (IID_IHciTimeout, (void**)&pTimeout);

.
// AddGroup, AddItems, acquire interface pointers for syncio timed and
// untimed
.

DWORD dwTimeoutVal = 1000; // 1 second
DWORD dwTransactionID = 0; // initial value

HANDLE hEvent = CreateEvent (NULL, FALSE, FALSE, NULL);
do
{
    hr = pSyncIOtimed->Read (pSyncIO,
                            dwTimeoutVal,
                            &dwTransactionID,
                            other params );

    if (hr == E_PENDING)
        WaitForSingleObject (hEvent, 100); // wait for 100
        milliseconds and try again

} while (hr == E_PENDING)

if (hr == E_ABORT)
{
    // Server is dead or hung
    // No cleanup required, decide how the client should proceed,
    // if it should proceed at all
}
else
{
    // handle other errors
    // retrieve results

```

5 HciTimeout Object

5.2 HciTimeout

```
        // cleanup memory or resources  
    }  
    .
```

CheckStatus syntax and description

```
STDMETHODIMP CheckStatus(DWORD dwTransactionID );
```

CheckStatus can be called to obtain the status of a pending operation. If a timed operation returns an HRESULT of E_PENDING, then CheckStatus provides a means to monitor the state of the operation. The transaction ID returned from the pending operation is used as the input. The status of the corresponding operation is returned in the HRESULT.

Another means to obtain the results from the Timeout object after the operation goes into a slow server state (E_PENDING) is to continue to call the original method with the returned transaction ID while monitoring the HRESULT. If the HRESULT changes from E_PENDING to a new condition, the operation is complete and the results are in the original out parameters, assuming the operation successfully completed.

Parameter Specifications

Table 9 CheckStatus Parameter Specifications

Parameters	Direction	Description
dwTransactionID	in	The input transaction ID must be the transaction ID returned from the original timed operation.

HRESULT Return Codes

Table 10 CheckStatus HRESULT Return Codes

Return Code	Description
S_OK	If this HRESULT is returned after an E_PENDING HRESULT, which resulted from the original method call or a previous CheckStatus call, this indicates that the slow server has completed and the results are available for the client.
E_PENDING	The server is slow. The client may wait for the server to complete its operation and retrieve the results once the HRESULTS transitions to S_OK. If the client decides to continue processing, it is still obligated to accept the future state of the server as well as freeing any memory or resources that may be returned from the server.
E_ABORT	The server is not responding (dead/hung). The client does not receive any results and should continue with other operations.
E_UNEXPECTED	Invalid transaction ID.
Other method specific HRESULTS.	See OPC specification 1.0A and HCI Client Developers Guide for a complete list of possible return codes.

Comments

This method can be called directly from the client; however, if the original method is recalled with the returned transaction ID, then the Timeout object makes this call on behalf of the client.

5 HciTimeout Object

5.2 HciTimeout

Sample code Custom

```
    IHciTimeout *pTimeout = NULL;
    CoCreateInstance(...);
    pUnk->QueryInterface(IID_IHciTimeout, (void**)&pTimeout);
    .
    // AddGroup, AddItems, acquire interface pointers for syncio timed and
    untimed
    .
    hr = pSyncIOTimed->Read(pSyncIO,
                           dwTimeoutVal,
                           &dwTransactionID,
                           ... other params );
    // event which may be signaled by thread upon completion
    HANDLE hEvent = CreateEvent(NULL, FALSE, FALSE, NULL);
    while (hr == E_PENDING)
    {
        hr = pTimeout->CheckStatus(dwTransactionID);
        if (hr != E_PENDING)
        {
            hr = pSyncIOTimed->Read(pSyncIO,
                                   dwTimeoutVal,
                                   &dwTransactionID,
                                   ... other params );
        }
        else
        {
            WaitForSingleObject(hEvent, dwTimeoutVal);
        }
    }
    .
    .
```

AnyTimedAutomationMethod

```
AnyTimedAutomationMethod(AppropriateObject as Object  
                          Timeout as long,  
                          TransID as long,  
                          original parameters )
```

Description

To use the Timeout object in VB, the client will need to include the appropriate reference. For OPC 1.0A clients, the typlib is “hcitimeout 1.0 Type Library (Ver 1.0)”. For OPC 2.0 clients, the typlib is “hcitimeout 2.0 Type Library (Ver 1.0)”. For automation clients, any timed call will contain the following three parameters in addition to the parameters normally required for that call. (To see the exact signature of the automation methods, include the hcitimeout type library located in the hcitimeout DLL).

The first parameter is the object on which the original call was to be made.

The second parameter is a long, which specifies how long to wait for the call to complete (in milliseconds).

The third parameter is a long, which will be the transaction ID for the call.

The automation client declares the timeout object corresponding to the OPC or HCI interface used. For example, IOPCSyncIODispTimed corresponds to IOPCSyncIODisp. The parameters to the timed call are the same except for the first three parameters. Automation clients can check the returned transaction ID to determine if the call completed in the specified time.

If a non-zero transaction ID is returned, then the call did not complete in the time specified. This means that the client should check the status of the transaction. This can be done by repeating the call with the returned transaction ID, or by calling the Status method. When the call does complete, the client can call the method again, to retrieve any returned parameters. If the operation did not complete because the server is dead, the timed call raises an error to notify the caller.

5 HciTimeout Object

5.2 HciTimeout

Parameter Specifications

Table 11 AnyTimedAutomationMethod Parameter Specifications

Parameters	Direction	Description
AppropriateObject	in	The object on which the original call is to be made.
Timeout	in	Number of milliseconds to wait for operation to complete before returning control to the client. 100 milliseconds is the minimum timeout period. If 1-99 is passed, then 100 milliseconds will be used. If 0 is passed, the operation will not timeout, it will wait until complete, just like an untimed OPC or HCI invocation. This is less efficient than an untimed OPC or HCI call. Do not pass a negative value for this parameter. If a negative value is passed, E_INVALIDARG will be returned.
TransID	in/out	The transaction ID should be a variable containing the value of 0. This implies that this is an initial invocation. For Synchronous OPCRead, Synchronous OPCWrite or GetStatus calls, the initial value can be 1. This indicates that if the server is not finished within the specified timeout, the client can proceed without regard to cleanup. The Timeout object cleans up the results and any outstanding memory from this invocation. If the server is slow and does not complete in the specified time period, control is returned to the client with an HciTimeout object generated transaction ID. This returned transaction ID is to be used to get the state of the operation and the results once the server finishes with the operation.
All other OPC parameters		

HRESULT Return Codes

Table 12 AnyTimedAutomationMethod HRESULT Return Codes

Return Code	Description
S_OK	If the transaction ID is zero, then the call was completed within the specified time.
S_FALSE	If the transaction ID is non-zero, then the server is slower than the specified time, but is expected to complete the operation.
E_ABORT	The server is not responding (dead/hung). The client does not receive any results and should continue with other operations.
E_POINTER	The interface pointer or Transaction ID pointer are invalid.
E_UNEXPECTED	An HciTimeout object error was detected. These errors include: Bad transaction ID. Returned values no longer available. Maximum number of outstanding threads exceeded.
E_OUTOFMEMORY	Client is out of memory.
Other method specific HRESULTS.	See OPC specification 2.0 and Hci Client Developers Guide for more information on possible return codes.

Automation Comments

The automation timeout calls differ from the custom calls, in that when the server is slow, no error is returned to VB clients. This avoids having to build complex error handling code just to determine if the call timed out. The VB client can check the transaction ID that is returned to determine if the call completed in time. If a call did not complete in time, the client can check on that transaction by either repeating the call with the same parameters until the transaction ID is set to zero, or by calling the Status method until S_OK (which is zero) is returned. If the server is dead, an error will be raised to the client. It is important to note that any optional parameters that are not specified the first time a timed call is made should not be specified when calling that call when the transaction has completed.

5 HciTimeout Object

5.2 HciTimeout

Sample Automation Code

If a group has been added, this code illustrates a timed OPCRead.

```
Private Sub Command1_Click()

Dim T As New HciTimeout
Dim TimedSyncIO As IOPCSyncIODispTimed
Dim transID As Long

'Set up the IOPCSyncIODispTimed interface
Set TsyncIO = T

NumItems = 2

'Normal syntax
'm_SyncIO.OPCRead NumItems, OPC_DS_DEVICE, _
    ItemSHandles, values, quals, , errorsread

'Timed syntax
transID = 0 'Initialize transaction ID

TsyncIO.OPCRead m_SyncIO, 3000, transID, NumItems, OPC_DS_DEVICE, _
    ItemSHandles, values, quals, , errorsread

'Non zero transaction ID means that it did not complete in time
If transID Then
    'Check the status until S_OK = 0 is returned
    While T.Status(transID)
        If MsgBox("Check the status again?", vbYesNo) =
vbNo Then
            Exit Sub
        End If
    Wend
End If

'Get the results by repeating the call, passing in the transaction ID
TsyncIO.OPCRead m_SyncIO, 3000, transID, 2, OPC_DS_DEVICE, _
    ItemSHandles, values, quals, , errorsread

End If

'Show the results
MsgBox values(1) & " " & quals(1)

End Sub
```

Status

```
STDMETHODIMP Status(long lTransID, long *pStatus);
```

Description

This method allows automation clients to check the status of a given transaction. The value returned in the pStatus parameter, is the HRESULT of the transaction specified.

Parameter Specifications

Table 13 Timeout Status Parameter Specifications

Parameters	Direction	Description
lTransID	in	The transaction ID that was returned from the timed automation call.
pStatus	returned value	The status of the given transaction. The possible results are: S_FALSE the transaction is still in progress. S_OK the transaction has completed. E_ABORT the transaction will not be completed.

Template Timeout Code

```
typedef map<DWORD, HRESULT> ID2HR;
typedef map<DWORD, void*> ID2Data;
ID2HR m_mpStatus; // map returning HR for each TransID
ID2HR::iterator m_mpStatusIter;
ID2Data m_mpData; // map containing pointer to data for each
ID2Data::iterator m_mpDataIter; // for TransID.
DWORD m_dwCurrentTransID;
BOOL m_bApartment;
CCriticalSection m_csTimeout;

const DWORD cdwCleanup = 1;
const DWORD cdwMaxThreads = 250;
const DWORD cdwMinTimeout = 100; // milliseconds
DWORD gdwOutstandingThreads;
void MyMethodThread(void * pvStruct);
const DWORD cdwMyMethod = 1;

struct MyMethodInfo
{
    DWORD dwMethodID;
    ChciTimeout *pThis;
    CCriticalSection *csTimeout;
    HANDLE hEvent; // Event to coordinate timeout
    DWORD dwTransID;
    BOOL bCleanupResults;
    IStream *pstInterfacePtr; // must marshal the
    // interface pointer if Apartment threaded
    IMyInterface *pMyInterface;
    HRESULT hResult; // HRESULT of Method
    TYPE1 param1;
    TYPE2 param2;
    HRESULT **ppErrors;
};

//-----
//-----
```

```

void MyMethodThread(void * pvStruct)
{
    HRESULT hr = S_OK;
    BOOL bApartment = FALSE;
    MyMethodInfo *pParams = (MyMethodInfo*)pvStruct;

    if (pParams->pstInterfacePtr)
    {
        // Apartment threaded
        bApartment = TRUE;
        CoInitialize(NULL);
        hr = CoGetInterfaceAndReleaseStream(pParams-
        >pstInterfacePtr,
        IID_IMyInterface,
        (void**) &(pParams->pMyInterface));
    }
    else
        // free threaded
        CoInitializeEx(NULL, COINIT_MULTITHREADED);

    gdwOutstandingThreads++;
    CHciTimeout *pThis = pParams->pThis; // ptr copy
    CcriticalSection *csTimeOut = pParams->csTimeOut; // ptr copy
    pParams->hResult = (pParams->pMyInterface)-
        >MyMethod(pParams->param1,
        pParams->param2,
        pParams->ppErrors);

    BOOL bServerNotResponding = FALSE; // server/instance may
        be dead
    if ((HRESULT_RPC(pParams->hResult)) ||
        (HRESULT_FACILITY(pParams->hResult) == FACILITY_RPC) ||
        ((pParams->hResult == E_OUTOFMEMORY) &&
        (bApartment))) bServerNotResponding = TRUE;

    // start lock - this lock will prevent false timeout. This
    means, if the
    // timing was such, that the mainline timer expired, just
    before the Event
    // was signaled.
    CSingleLock guard(csTimeOut);
    guard.Lock();
    hr = pThis->CheckStatus(pParams->dwTransID);

    if (hr == E_PENDING) // timeout occurred in mainline
    {
        if (bServerNotResponding)
        {
            hr = E_ABORT; // server is dead
            if (pParams)
            {
                pThis->SetStatus(pParams->dwTransID, hr);
                delete pParams;
                pParams = NULL;
            }
        }
        else // server is slow
        {
            // client doesn't want results if server is slow
            if (pParams->bCleanupResults)
            {

```

5 HciTimeout Object

5.2 HciTimeout

```
        if (SUCCEEDED(pParams->hResult))
        {
            CoTaskMemFree(pParams->ppErrors);
            pParams->ppErrors = NULL;
        }

        pThis->RemoveTransID(pParams->dwTransID);
        delete pParams;
        pParams = NULL;
    }
    else
    {
        // copy state to map
        pThis->CopyData(pParams->dwTransID,
            (void*)pParams);
        hr = S_OK;
        pThis->SetStatus(pParams->dwTransID, hr);
    }
}
else // server is alive and fast
{
    if (bServerNotResponding)
        hr = E_ABORT;
    else
    {
        hr = S_OK;
        pThis->SetStatus(pParams->dwTransID, hr);
        SetEvent(pParams->hEvent);
    }
}
guard.Unlock();
if (csTimeOut)
{
    delete csTimeOut;
    csTimeOut = NULL;
}
dynamic_cast<IHciTimeout*> (pThis)->Release();
gdwOutstandingThreads--;
CoUninitialize();
} // end thread

//-----
-----

STDMETHODIMP CMyTimeout::MyMethod(IMyInterface *pIf,
    DWORD dwTimeout,
    DWORD *dwTransID,
    TYPE1 param1,
    TYPE2 param2,
    HRESULT **ppErrors)
{
    HRESULT hr = E_FAIL;

    if ((!pIf) || (!dwTransID))
        return E_POINTER;

    MyMethodInfo *pParams = NULL;

    // Cleanup Results if Timeout only valid for some methods
    BOOL bCleanupResults = FALSE;
    if (*dwTransID == cdwCleanup)
    {
```

```

        *dwTransID = 0;
        bCleanupResults = TRUE;
    }
    if (*dwTransID) // must be a request for data
that has already been spawned
    {
        hr = CheckStatus(*dwTransID);
        if (hr == S_OK)
        {
            m_mpDataIter = m_mpData.find(*dwTransID);
            if (m_mpDataIter != m_mpData.end())
            {
                pParams =
(MyMethodInfo*)m_mpData[*dwTransID];
                if (pParams)
                {
                    if (pParams->dwMethodID == cdwMyMethod)
                    {
                        param1 = pParams->param1; // retrieve result values
                        param2 = pParams->param2;
                        ppErrors = pParams->ppErrors; // retrieve error array
                        hr = pParams->hResult; // method results
                        delete pParams;
                        pParams = NULL;

                        RemoveTransID(*dwTransID);
                        m_mpData.erase(*dwTransID);
                    }
                    else
                        hr = E_UNEXPECTED;
                    // wrong trans ID
                }
            }
            else
                hr = E_UNEXPECTED;
        }
        else
            hr = E_UNEXPECTED;
    }
    return hr;
}

if (gdwOutstandingThreads >= cdwMaxThreads)
    return E_UNEXPECTED;

// New request, Initialize timed request status
*dwTransID = GetTransID();
HANDLE *hEvent = new HANDLE;
*hEvent = CreateEvent(NULL, FALSE, FALSE, NULL);

// do we need to support apartment thread model or always
free ?????
pParams = new MyMethodInfo; // Create copy of
                             Params struct on the heap
                             // to pass between threads
if (!pParams)
    return E_OUTOFMEMORY;

CCriticalSection *csTimeOut = new CCriticalSection;

pParams->pThis = this;
dynamic_cast<IHciTimeout*> (this)->AddRef();

```

5 HciTimeout Object

5.2 HciTimeout

```
        if (m_bApartment)
            hr =
                CoMarshalInterThreadInterfaceInStream(IID_IMyInterface,
                                                       pIf,
                                                       &(pParams->pstInterfacePtr));
        else
            pParams->pstInterfacePtr = NULL;

        pParams->dwMethodID          = cdwMyMethod;
        pParams->csTimeout            = csTimeout;
        pParams->hEvent               = *hEvent;
        pParams->bCleanupResults      = bCleanupResults;
        pParams->pMyInterface         = pIf;
        pParams->dwTransID            = *dwTransID;
        pParams->hResult              = E_PENDING;
        pParams->param1               = param1;
        pParams->param2               = param2;
        pParams->ppErrors              = ppErrors;

        if (dwTimeout == 0)
            MyMethodThread ((void*)pParams);
        else
        {
            if (dwTimeout < cdwMinTimeout)
                dwTimeout = cdwMinTimeout;
            HANDLE hThread = (HANDLE)::_beginthread(MyMethodThread,
                                                    0, (void*)pParams);
        }

        // use timed event
        DWORD dwEventStatus = WaitForSingleObject(*hEvent, dwTimeout);
        if (dwEventStatus == WAIT_OBJECT_0)
        {
            hr = CheckStatus(*dwTransID);
            // No timeout
            if (pParams)
            {
                if (hr != E_ABORT)
                {
                    // retrieve result values
                    param1 = pParams->param1;
                    param2 = pParams->param2;
                    ppErrors = pParams->ppErrors; // retrieve error array
                    hr = pParams->hResult; // method results
                }
                delete pParams;
                pParams = NULL;
            }

            RemoveTransID(*dwTransID);
            *dwTransID = 0;
            if (hEvent)
            {
                delete hEvent;
                hEvent = NULL;
            }
        }
        else
        {

```

```
// timeout occurred
CSingleLock guard(csTimeOut, TRUE); // used
                                     with corresponding thread,
                                     // scope unlocks it
hr = CheckStatus(*dwTransID);
if (hr == S_OK)                      // method complete
{
    // timing of thread was so close, that it
    // really finished ok
    if (pParams)
```

5 HciTimeout Object

5.2 HciTimeout

```
param1 = pParams->param1;    {
                             // retrieve result values
                             param2 = pParams->param2;
ppErrors      = pParams->ppErrors; // retrieve error array
                             hr = pParams->hResult; // method
                                                results

                             delete pParams;
                             pParams = NULL;

                             RemoveTransID(*dwTransID);
                             m_mpData.erase(*dwTransID);
                             *dwTransID = 0;
                             }
else
{
    hr = E_PENDING; // timeout
    SetStatus(*dwTransID, hr);
}

if (hEvent)
{
    delete hEvent;
    hEvent = NULL;
}
// unlocked by scope
}

return hr;
}
```

6. Custom Interface APIs

6.1 Overview

Introduction

This section serves as a reference for the C++ programmer using the OPC Custom interface together with the HCI client library, which provide additional development features.

6.2 HCI Class

Functional description

This class contains static methods useful to HCI client and application developers. These methods allow an HCI client to easily obtain the required information necessary to interface with an HCI Component object.

Public Methods list

Below is a list of the Public Methods that can be used. Detailed descriptions of each them follows.

- GetComponentInfo
- GetCurrentDomainName
- GetLocalHostName
- IsLocalHost

6 Custom Interface APIs

6.2 HCI Class

GetComponentInfo

```
HRESULT HCI::GetComponentInfo (LPCWSTR pwsComponentName,  
                              CLSID *clsidClass,  
                              Cstring *pszProgID,  
                              Cstring *pszHostName);
```

Description

Returns information about the specified HCI component. Used to obtain information about an HCI component, which is needed for CoCreateInstanceEx().

Parameter Specifications

Table 14 GetComponentInfo Parameter Specifications

Parameters	Direction	Description
pwsComponentName	in	Pointer to the component name.
clsidClass	out	Pointers to a variable of type CLSID that will receive the class ID of the component. If NULL, the class ID is not returned.
pszProgID	out	Pointer to a CString objects that will receive the ProgID of the component. If NULL, the ProgID is not returned.
pszHostName	out	Pointer to a CString object that will receive the hostname where the component resides. If NULL, the hostname is not returned.

HRESULT Return Codes

Table 15 GetComponentInfo HRESULT Return Codes

Return Code	Description
S_OK	The function was successful.
E_FAIL	The function was unsuccessful.

Comment

None.

GetCurrentDomainName

```
HRESULT HCI::GetCurrentDomainName(CString &szDomainName);
```

Description

Returns the TPS domain name in which the caller is running. An example of a returned domain name is /TPS100TESTDOMAIN. This method looks up the TPS domain name from the current node's registry under HKEY_LOCAL_MACHINE\Honeywell\MyTpsDomain hive and stores it in local cache, thereby avoiding the expense of going out to the registry each time it is called.

Parameter Specifications

Table 16 GetCurrentDomainName Parameter Specifications

Parameters	Direction	Description
szDomainName	out	TPS Domain name

HRESULT Return Codes

Table 17 GetCurrentDomainName HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
S_INVALIDARG	One or more arguments was invalid
E_FAIL	the function was not successful

Comments

None.

6 Custom Interface APIs

6.2 HCI Class

GetLocalHostName

```
HRESULT GetLocalHostName(CString &szHostName);
```

Description

Returns the local host name all in uppercase. An example of a returned host name is TPS100TEST1. Note that this method is optimized to locally cache the hostname that is returned from a Windows system call, thereby avoiding the expense of a system call each time GetLocalHostName is called.

Parameter Specifications

Table 18 GetLocalHostName Parameter Specifications

Parameters	Direction	Description
szHostName	out	Host name

HRESULT Return Codes

Table 19 GetLocalHostName HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
E_INVALIDARG	One or more arguments was invalid
E_FAIL	The function was unsuccessful

Comments

None.

IsLocalHost

```
HRESULT HCI::IsLocalHost (LPCWSTR szHostName);
```

Description

Returns S_OK if szHostName is the local host name. Returns S_FALSE if szHostName is not the local hostname.

Parameter Specifications

Table 20 IsLocalHost Parameter Specifications

Parameters	Direction	Description
szHostName	in	Host name

HRESULT Return Codes

Table 21 IsLocalHost HRESULT Return Codes

Return Code	Description
S_OK	szHostName is the local host name
S_FALSE	szHostName is not the local host name
E_FAIL	the function was not successful

Comments

None.

6.3 IHciClientUtil

Functional description

This interface contains methods useful to HCI client and application developers. These methods allow an HCI client to easily obtain the required information necessary to interface with an HCI Component object.



ATTENTION

IHciClientUtil is the COM version of the Hci class, which is currently available in the static library, hciutil.lib. The Honeywell Communications Interface is providing some client interfaces as COM objects in order to provide Release Independence from previous HCI releases. IHciClientUtil is an interface of HciClientUtil COM object. This COM object is provided within clientutil.dll. The names and functionality of the methods are the same as those provided by Hci class. However, the method signatures are modified in order to be COM compliant. Use of IHciClientUtil methods requires connection to the inproc server HciClientUtil.

Connecting to HciClientUtil COM object

In order to use IHciClientUtil, use CoCreateInstance to connect to the inproc server HciClientUtil. You must include clientutil_i.c and clientutil.h in your source file. The following sample code illustrates how to connect to the HciClientUtil inproc server.

```
IHciClientUtil *pHciClientUtil = NULL
hr = CoCreateInstance(CLSID_HciClientUtil,
                     NULL,
                     CLSCTX_INPROC_SERVER,
                     IID_IHciClientUtil,
                     (void **)&pHciClientUtil);
```

Public Methods Summary

- GetComponentInfo
- GetCurrentDomainName
- GetLocalHostName
- IsLocalHost
- HRESULTToString

GetComponentInfo

```
STDMETHODIMP GetComponentInfo (LPCWSTR pwsComponentName,
                               CLSID *pclsidClass,
                               LPWSTR *ppszProgID,
                               LPWSTR *ppszHostName) ;
```

Description

Returns information about the specified HCI component. Used to obtain information about an HCI component, which is needed for CoCreateInstanceEx().

Parameter Specifications

Table 22 GetComponentInfo Parameter Specifications

Parameters	Direction	Description
pwsComponentName	in	Pointer to the component name.
pclsidClass	out	Pointer to a variable of type CLSID that will receive the class ID of the component. If NULL, the class ID is not returned.
ppszProgID	out	Pointer to a wide string pointer that will receive the ProgID of the component. If NULL, the ProgID is not returned.
ppszHostName	out	Pointer to a wide string pointer that will receive the hostname where the component resides. If NULL, the hostname is not returned.

HRESULT Return Codes

Table 23 GetComponentInfo HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was unsuccessful
E_INVALIDARG	The specified component name is invalid

Comments

If requested, the caller must free the ProgID and HostName returned in ppszProgID and ppszHostName.

6 Custom Interface APIs

6.3 IHciClientUtil

GetCurrentDomainName

```
STDMETHODIMP GetCurrentDomainName(LPWSTR *ppszDomainName);
```

Description

Returns the TPS domain name in which the caller is running. An example of a returned domain name is /TPS100TESTDOMAIN. This method looks up the TPS domain name from the current node's registry under the
HKEY_LOCAL_MACHINE\Honeywell\MyTpsDomain
hive and stores it in local cache, thereby avoiding the expense of going out to the registry each time it is called.

Parameter Specifications

Table 24 GetCurrentDomainName Parameter Specifications

Parameters	Direction	Description
ppszDomainName	out	TPS Domain name

HRESULT Return Codes

Table 25 GetCurrentDomainName HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
S_INVALIDARG	One or more arguments was invalid
E_FAIL	the function was not successful

Comments

The caller must free the domain name returned in ppszDomainName.

GetLocalHostName

```
STDMETHODIMP GetLocalHostName (LPWSTR *ppszHostName) ;
```

Description

Returns the local host name all in uppercase. An example of a returned host name is TPS100TEST1. Note that this method is optimized to locally cache the hostname that is returned from an NT System call, thereby avoiding the expense of a system call each time GetLocalHostName is called.

Parameter Specifications

Table 26 GetLocalHostName Parameter Specifications

Parameters	Direction	Description
ppszHostName	out	Host name

HRESULT Return Codes

Table 27 GetLocalHostName HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
E_INVALIDARG	One or more arguments was invalid
E_FAIL	The function was unsuccessful

Comments

The caller must free the host name returned in ppszHostName.

6 Custom Interface APIs

6.3 IHciClientUtil

IsLocalHost

```
STDMETHODIMP IsLocalHost (LPCTSTR szHostName);
```

Description

Returns S_OK if szHostName is the local host name. Returns S_FALSE if szHostName is not the local hostname.

Parameter Specifications

Table 28 IsLocalHost Parameter Specifications

Parameters	Direction	Description
szHostName	in	Host name

HRESULT Return Codes

Table 29 IsLocalHost HRESULT Return Codes

Return Code	Description
S_OK	szHostName is the local host name
S_FALSE	szHostName is not the local host name
E_FAIL	the function was not successful

Comments

None.

HresultToString

```
STDMETHODIMP HresultToString (HRESULT hr,
                              LPWSTR *ppszString );
```

Description

HresultToString is a method of the HciClientUtil COM object. It will return a string that corresponds to the passed in HRESULT.

Parameter Specifications

Table 30 HresultToString Parameter Specifications

Parameters	Direction	Description
hr	in	HRESULT to be looked up and converted to a string.
ppszString	out	String representing the HRESULT passed into this method. The client is responsible for freeing up this string with CoTaskMemFree.

HRESULT Return Codes

Table 31 HresultToString HRESULT Return Codes

Return Code	Description
S_OK	szHostName is the local host name
E_OUTOFMEMORY	Not enough memory
E_FAIL	the function was not successful because the specified HRESULT was not found

Comments

The client is responsible for freeing the returned string with CoTaskMemFree.

6.4 IHciAttributes Interface

Functional Description

The IHciAttributes interface defines a set of operations related to server attributes, such as priorities, access level, and so forth. IHciAttributes is on the device specific server portion of the HCI Server. Clients using C++ use QueryInterface to obtain a pointer to this interface, while automation clients must access the object through the CCHciVBHelper class. The specific HCI server provides the actual attribute support. To locate the following information, the client developer is referred to the user's guide that relates to the target HCI server:

- Definition and/or configuration of server attribute values.
- Names of attributes supported. These names are required as input parameters to get and set methods.
- Associated security levels defined per attribute supported.

Attribute settings are in effect for only the server instance that is connected to that particular client.

A specified server attribute value remains in effect until the client requests to set a new value for the same server attribute. If the client does not explicitly request to set attribute values, all requests are executed at the default attribute values established by the HCI Server.

The ability to set server attributes can be secured at the attribute level. The HCI Server establishes this security.

Public Methods Summary

- SetAttributes
- GetAttributes

SetAttributes

```
STDMETHODIMP SetAttributes (DWORD dwNumAttr,
                           LPCWSTR *pszAttrNames,
                           VARIANT *pvAttrValues,
                           HRESULT **ppErrors);
```

Description

None.

Parameter Specifications

Table 32 SetAttributes Parameter Specifications

Parameters	Direction	Description
dwNumAttr	in	The number of server attributes in the array that are passed in, which is the number of elements in the returning arrays.
pszAttrNames	in	The list of server attribute names to be set.
pvAttrValues	in	The list of server attribute values to be set.
ppErrors	out	Array of HRESULTs indicating the success of the individual server attribute sets. The errors correspond to the names passed in pszAttrNames. This indicates whether the HCI Server accepted the new value of the server attribute. NOTE any FAILED error code indicates the HCI Server rejected the value.

HRESULT Return Codes

Table 33 SetAttributes HRESULT Return Codes

Return Code	Description
S_OK	The function was successful.
S_FALSE	The function was partially successful. See ppErrors to determine server attribute specific errors.
E_INVALIDARG	An invalid argument is being used.
E_FAIL	The function was unsuccessful.

ppErrors Return Codes

Table 34 SetAttributes ppErrors Return Codes

ppErrors Return Code	Description
S_OK	The attribute was successfully set.
E_ACCESSDENIED	The specific HCI Server has determined that the caller does not have sufficient access rights to set the specified server attribute.
E_NOTIMPL	The specific HCI Server does not support the specified server attribute.
E_FAIL	The specific HCI Server was unable to set the specified server attribute.

Comment

None.

GetAttributes

```
STDMETHODIMP GetAttributes (DWORD dwNumAttr,  
                             LPCWSTR *pszAttrNames,  
                             VARIANT **ppvAttrValues,  
                             HRESULT **ppErrors);
```

Description

GetAttributes retrieves the current value associated with the server attributes specified in pszAttrNames.

Parameter Specifications

Table 35 GetAttributes Parameter Specifications

Parameters	Direction	Description
dwNumAttr	in	The number of server attributes in the array that are passed in, which is the number of elements in the returning arrays.
pszAttrNames	in	The list of server attribute names to be retrieved.
ppvAttrValues	out	The list of the current values of server attributes specified by pszAttrNames.
ppErrors	out	Array of HRESULTs indicating the success of the individual server attribute gets. The errors correspond to the names passed in pszAttrNames.

HRESULT Return Codes

Table 36 GetAttributes HRESULT Return Codes

Return Code	Description
S_OK	The function was successful.
S_FALSE	The function was partially successful. See ppErrors to determine server attribute specific errors.
E_INVALIDARG	An invalid argument is being used.
E_FAIL	The function was unsuccessful.

ppErrors Return Codes

Table 37 GetAttributes ppErrors Return Codes

ppErrors Code	Description
S_OK	The attribute was successfully retrieved.
E_ACCESSDENIED	The specific HCI Server has determined that the caller does not have sufficient access rights to get the specified server attribute.
E_NOTIMPL	The specific HCI server does not support the specified server attribute.
E_FAIL	The specific HCI server was unable to get the specified server attribute.

Comment

None.

6.5 IHciPulse Interface

Purpose

The IHciPulse interface is available to clients to monitor the state of an IAdviseSink connection between the server and client. It is possible to lose an IAdviseSink connection as a result of heavy network traffic or broken physical connections. Although an RPC exception is raised in the server, the exception cannot be caught by an exception handler. Furthermore, since the IAdviseSink::OnDataChange method has no return status, the server is not able to determine if the OnDataChange call to the client succeeded or failed. Therefore, the server is likely to continue to send data, such as OnDataChange notifications to the client. Since the client can still communicate with the server, it continues to listen for OnDataChange notifications but receives nothing since the IAdviseSink connection is lost. From the client's perspective, this scenario translates to a potential loss of data. IHciPulse is specifically provided for those clients that implement OnDataChange with OPC 1.0A mechanism. This interface is not necessary for clients that implement OnDataChange with OPC 2.0 mechanisms.

Functional description

The IHciPulse interface is a mechanism to send a ping from the server to the client on the same "channel" as the OnDataChange. It has methods to configure, start, stop and retrieve information regarding the pulse. C++ clients may establish a pulse on each IAdviseSink connection that it wishes to monitor. Understood that IAdviseSink connections are established on a group basis but it is possible to have more than one connection per group. VB clients can only have one connection per group. In order to use the IHciPulse interface effectively, the client performs three steps:

1. Configure the pulse to occur on a periodic basis.
2. Monitor the pulse notifications that are received from the server. The pulse notifications are broadcast to the OnDataChange.
3. Take corrective action if a pulse notification is not received by the client for several successive periods. A typical client side corrective action: "Re-sink" to the server. Re-sink is the process of closing the current connection associated with the IAdviseSink and reestablishing a new IAdviseSink connection.

A custom client can distinguish a pulse transmission from a normal data transmission by validating the transaction ID in the group header of the stream against the transaction ID, which was defined when the pulse was configured. The two transaction ID's must match. Additionally, the number of items defined in the group header of the stream will be zero and the HRESULT in the group header will be S_OK.

6 Custom Interface APIs

6.5 IHciPulse Interface

An automation client can distinguish a pulse transmission from a normal data transmission if the count parameter of the OnDataChange is equal to zero.

The IHciPulse interface is directly available for clients which utilize the OPC custom interfaces and indirectly available for automation or VB clients through the HCI VB helper object. Timed versions (Hci Timeout object) of the IHciPulse interface are also available for both custom and automation clients.

Public Methods summary

The IHciPulse interface provides the following methods

- Configure
- CancelPulse
- Start
- GetInfo

Configure

```
STDMETHODIMP Configure(DWORD dwCycles,
                       DWORD dwConnection,
                       BOOL bStart,
                       DWORD dwTransID);
```

Description

This method is called to establish and optionally start a single Hci pulse. Once configured and started, the server sends the pulse every period. The period is equal to the number of cycles times the update rate of the group. One cycle is equal to the update rate of the group.

Typically, the client would not want to flood the network with pulses at the same rate for which the OnDataChanges occur. If the update rate is very high, greater than 1 hour for instance, then it may be feasible to have the pulse cycle equal to 1. In general, if the update rate of an active group is several seconds, then the number of cycles should be greater than or equal to 5. This will monitor the OnDataChange “channel” and alleviate network traffic.

A group with a 4-second update rate and a pulse configured for five cycles would translate to a pulse from the server to the client at a period of approximately every 20 seconds. If the client misses three or more successive periods, then the client may want to re-sink the IAdviseSink connection. The same connection obtained from the DAdvise on a group must be used as the connection when configuring the pulse.

The client can choose to set the pulse to start as part of the configuration. Use of this option saves one round trip to the server assuming that the client wants the pulse started after it is configured. A unique transaction ID, assigned by the client, is part of the

configuration. This unique transaction ID is returned in the group header of the stream for each pulse. This is used to distinguish a pulse notification from a normal onDataChange notification.

Parameter Specifications

Table 38 Configure Parameter Specifications

Parameters	Direction	Description
dwCycles	in	The number of cycles between pulses sent from the server to the client. The length of a cycle is equal to the update rate of the group.
dwConnection	in	Connection key established for the DAdvise. Potentially one for each IAdviseSink.
bStart	in	If bStart is TRUE then the Pulse will be started automatically, otherwise if the value is FALSE, then the Pulse will not be started until the Start method is called. FALSE will not cause a CancelPulse if currently running.
dwTransID	in	Unique value provide by the client. This value is also returned in the Transaction ID field of the group header in the stream. It is used to identify if the packet received is from a pulse or not.

HRESULT Return Codes

Table 39 Configure HRESULT Return Codes

Return Code	Description
S_OK	The function was successful.
S_FALSE	The function was successful. The cycles were modified from a previous configuration.
E_FAIL	The function was unsuccessful. Callback has not been setup via DAdvise.
E_INVALIDARG	The function was unsuccessful. The number of cycles = zero.

6 Custom Interface APIs

6.5 IHciPulse Interface

Start

```
STDMETHODIMP Start(DWORD dwConnection);
```

Description

This method is called to start a configured pulse. The parameter, dwConnection, is used to determine which pulse to start. The connection is obtained when the client creates a connection through the DAdvise.

Parameter Specifications

Table 40 Start Parameter Specifications

Parameters	Direction	Description
dwConnection	in	The connection obtained when setting up the DAdvise.

HRESULT Return Codes

Table 41 Start HRESULT Return Codes

Return Code	Description
S_OK	The function was successful.
S_FALSE	The function was successful. The connection does not exist.

Comments

None.

CancelPulse

STDMETHODIMP CancelPulse (DWORD dwConnection);

Description

This method is called to explicitly stop a particular Hci pulse. If a group is set from active to inactive, then all pulses on that group will be stopped, just as if CancelPulse was called for each pulse on that group.

Parameter Specifications

Table 42 CancelPulse Parameter Specifications

Parameters	Direction	Description
dwConnection	in	Connection key established for the DAdvise. Potentially one for each IAdviseSink.

HRESULT Return Codes

Table 43 CancelPulse HRESULT Return Codes

Return Code	Description
S_OK	The function was successful.
S_FALSE	The function was successful. No callback channel exists.

Comments

None.

GetInfo

```
STDMETHODIMP GetInfo (DWORD *pdwNumConnections,  
    BOOL *pbActive,  
    DWORD *pdwUpdateRate,  
    DWORD *pdwCycles,  
    DWORD **ppTransIDs,  
    DWORD **ppConnections);
```

Description

This method is called to retrieve the current configuration of all pulses associated with a group. The information will include the current number of pulses configured for the group, the activeness of the group, the group update rate, the number of cycles between pulses, an array of the transaction ids (1 for each configured pulse) and an array of the connections (1 for each configured pulse). As with all out parameters, the client is responsible for releasing the memory associated with the out parameters. This includes releasing the memory for the contents of the returned arrays.

Parameter Specifications

Table 44 GetInfo Parameter Specifications

Parameters	Direction	Description
*pdwNumConnections	out	The current number of pulses associated with the group
*pbActive	out	If the group is active, then TRUE is returned, otherwise FALSE is returned.
*pdwUpdateRate	out	The current update rate of the group in milliseconds.
*pdwCycles	out	The number of cycles between pulses. One cycle is equivalent to the update rate.
**ppTransIDs	out	An array of the transaction ids. There is a unique transaction ID for each pulse that was generated by the client.
**ppConnections	out	An array of connections which were created when the client perform the DAdvise. There will be a unique connection for each IAdviseSink that was setup by the client.

HRESULT Return Codes

Table 45 GetInfo HRESULT Return Codes

Return Code	Description
S_OK	The function was successful.
E_FAIL	The function was unsuccessful.
E_INVALIDARG	The function was unsuccessful. Could be one of several problems. An invalid pointer was passed to the method or the number of configured pulses is zero.
E_OUTOFMEMORY	The function was unsuccessful. The server process ran out of memory.

6.6 HciSink Class

Purpose

The purpose of this class is to handle IAdviseSink calls for the client. OnDataChange is used as the asynchronous mechanism. Dadvise and Dunadvise have been provided so that a callback function, which is defined by the client, can be used with the AsyncIO interface defined in OPC 1.0a. Clients would *not* use this call with AsyncIO2 provided in OPC 2.0.

Functional Description

It enables objects to receive notifications of data changes, view changes, and receive asynchronous data through callbacks. HciSink is an *optional* client side utility class that encapsulates the IDataObject DAdvise, Dunadvise, and OnDataChange.

6 Custom Interface APIs

6.6 HciSink Class

Public Methods Summary

The HciSink class provides the following public methods

- HciSink
- ~HciSink
- DAdvise (2 versions)
- DUnadvise
- OnDataChange
- GetDataObject
- SetDataObject
- QueryInterface
- AddRef
- Release

HciSink

```
HciSink(IDataObject *pDataObj);
```

Description

Constructor.

Parameter Specifications

Table 46 HciSink Parameter Specifications

Parameters	Direction	Description
pDataObj	in	A pointer to an IDataObject with which IAdviseSink relationships will be established.

Comments

None.

~HciSink

```
~HciSink( );
```

Description

Destructor.

Parameter Specifications

None

Comments

HciSink user must perform a Release() when the HciSink is no longer needed. Note that even if the object was constructed on the heap using New, a Release() is required. The Release() will invoke the “Delete.”

6 Custom Interface APIs

6.6 HciSink Class

Dadvise

```
HRESULT DAdvise(void (*pCallback) (HRESULT, IStream *),  
                OPCSTREAMFORMAT fmt,  
                DWORD *pdwConnection = NULL);
```

Description

DAdvise is provided to hide the FORMATETC from the client and provide the ability to have a callback routine. Normally the client would call the server's IDataObject::Dadvise method; however, it is forwarded to the server from here. Once the server has serviced the request and packaged the data in a stream, it forwards the results to HciSink::OnDataChange. OnDataChange checks to see if the format is valid. If the format is valid, then the local callback is invoked with an HRESULT of S_OK.

Parameter Specifications

Table 47 Dadvise Parameter Specifications

Parameters	Direction	Description
pCallback	in	A pointer to a local callback function (cannot be a class method) where the client application wishes to receive data change notifications.
fmt	in	Stream format. Refers to one of three stream formats used in the IDataObject/IAdviseSink connection. Refer to Data Stream Formats in OPC Specification for details.
pdwConnection	out	A pointer to a DWORD that will be updated with the connection identifier.

HRESULT Return Codes

Table 48 Dadvise HRESULT Return Codes

Return Code	Description
S_OK	The advisory connection was created.
E_OUTOFMEMORY	Not enough memory
E_INVALIDARG	One or more arguments was invalid.
E_UNEXPECTED	An unexpected error occurred.
E_FAIL	The function was unsuccessful.
HCI_E_BADSTREAM FORMAT	Bad stream format

Comments

None

6 Custom Interface APIs

6.6 HciSink Class

Dunadvise

```
HRESULT Dunadvise( );
```

Description

DUnadvise is called when it is desired to turn off the asynchronous updates.

Parameter Specifications

None

HRESULT Return Codes

Table 49 Dunadvise HRESULT Return Codes

Return Code	Description
S_OK	The function was successful.
S_FALSE	There is nothing to enumerate (There are no items in the group).
E_OUTOFMEMORY	Not enough memory
E_INVALIDARG	A bad riid parameter was passed.
E_UNEXPECTED	An unexpected error occurred.
E_FAIL	The function was unsuccessful.

Comments

None

OnDataChange

```
void OnDataChange(FORMATETC *pfmtetc,  
                  STGMEDIUM *pstg);
```

Description

Standard DCOM IAdviseSink::OnDataChange method. See Visual C++ or Win32 help for more information.

Comments

The HciSink class implements this method to support TPS data access. Users of the HciSink class do not call this method directly. Instead, the component object with which the client application has established an IAdviseSink relationship invokes this method to cause a client callback.

GetDataObject

```
IDataObject* GetDataObject(void)
```

Description

This method returns a pointer indicating the IDataObject interface is currently in use by the HciSink object. This interface is normally internal to HciSink and completely encapsulated; however, when retrofitting an HciSink object into an apartment model application, it may be necessary to call HciSink::DAdvise() and HciSink::DUnadvise from separate threads. In this case, the IDataObject interface pointer must be marshalled between the threads. Use this method to obtain the current interface pointer to be marshalled.

Comments

This method is only useful for marshalling the internal IDataObject interface between threads of an apartment model application.

SetDataObject

```
void SetDataObject(IDataObject*pMarshaledObject)
```

Description

This method allows an application to set the internal IDataObject interface pointer. This interface is normally internal to HciSink and completely encapsulated. However, when retrofitting an HciSink object into an apartment model application, it may be necessary to call HciSink::DAdvise() and HciSink::DUnadvise from separate threads. In this case, the IDataObject interface pointer must be marshalled between the threads.



ATTENTION

This method must only be used for marshalling this IDataObject interface pointer between the threads of an apartment threading model application. The pMarshaledObject parameter must refer to the exact same interface as the current IDataObject interface.

Parameter Specifications

Table 50 SetDataObject Parameter Specifications

Parameters	Direction	Description
pMarshaledObject	in	A pointer to an IDataObject interface. This pointer must be the original IDataObject interface encapsulated by this HciSink object marshalled to a different thread.

Comments

This method is only useful for marshalling the internal IDataObject interface between threads of an apartment model application. Obtain the current IDataObject interface through HciSink::GetDataObject(). Marshal the interface to the desired thread. Assign the marshalled interface back into the HciSink using this method.

QueryInterface

```
HRESULT QueryInterface(REFIID riid,  
                        void **ppv);
```

Description

Standard DCOM IUnknown::QueryInterface method. See Visual C++ or Win32 help for more information.

Comments

None.

AddRef

```
ULONG AddRef( );
```

Description

Standard DCOM IUnknown::AddRef method. See Visual C++ or Win32 help for more information.

Comments

None.

Release

```
ULONG Release( );
```

Description

Standard DCOM IUnknown::Release method. See Visual C++ or Win32 help for more information.

Comments

Release must be called when the HciSink object is no longer needed, even if the object was created on the heap using "New." This Release is required rather than "delete." A Release() should also be performed to match all AddRefs(). See also ~HciSink.

6.7 HciStreamUtil Class

Purpose

Utility class retrieves data from a given stream into the OPC stream format. Use of HciStreamUtil is made available for use by a client to unpack the data stream received as a notification through IAdviseSink::OnDataChange. Its use is optional. However, if a clients elects to provide an alternative utility to unpack the stream, it must adhere to the exact format as defined by OPC. This class should only be used by OPC 1.0A clients who are using IAdviseSink. OPC 2.0 clients would *not* use this class since IOPCDataCallback does not use streams.

Functional Description

OPC has three different stream formats (refer to Client Side Interfaces and Data Stream Formats sections in OPC specification). On Data Change, Refresh, and Asynchronous Read has 2 formats: OPCSTMFORMATDATA and OPCSTMFORMATDATATIME. The asynchronous write has a single format: OPCSTMFORMATWRITECOMPLETE. Refer to the Data Stream Formats section in the OPC specification for a description of the data structures associated with the three stream formats. The data in the stream consists of a group header followed by one or more item headers followed by the data. The HciStreamUtil class is used to retrieve the data, group header, item count, stream format, or transaction ID. Data can be retrieved from the stream by index or handle.

Public Methods summary

- HciStreamUtil
- ~HciStreamUtil
- GetByIndex
- GetByHandle
- GetGroupHeader
- GetItemCount
- GetStreamFormat
- GetTransID

HciStreamUtil

```
HciStreamUtil(IStream &strm, OPCSTREAMFORMAT fmt));
```

Description

Constructor who requires an IStream as input to construct this object. Stream must be used according to the format specified. The streamutil is initialized with the specified format. All subsequent operations will assume the stream is in the “constructed” format. Failure to conform will result in non-deterministic problems.

Parameter Specifications

Table 51 HciStreamUtil Parameter Specifications

Parameters	Direction	Description
strm	in	The IStream must not be NULL, it must be a valid IStream.
fmt	in	Stream format. Refers to one of three stream formats used in the IDataObject/IAdviseSink connection. Refer to Data Stream Formats in OPC Specification for details.

HRESULT Return Codes

None

Comments

None

6 Custom Interface APIs

6.7 HciStreamUtil Class

~HciStreamUtil

```
~HciStreamUtil( );
```

Description

Destructor

Parameter Specifications

None

HRESULT Return Codes

None

Comments

None

GetByIndex (for OPCSTMFORMATDATETIME)

```
STDMETHODIMP GetByIndex (DWORD      dwIndex,
                          OPCHANDLE  &hClientGroup,
                          OPCHANDLE  &hClient,
                          WORD        &wQuality,
                          FILETIME    &ftTimeStampItem,
                          VARIANT &vtValue);
```

Description

GetByIndex is used to retrieve data from the stream by an index into the stream of the format OPCSTMFORMATDATETIME. Use GetItemCount or GetGroupHeader to determine the number of items returned in the stream.

Parameter Specifications

Table 52 GetByIndex (for OPCSTMFORMATDATETIME) Parameter Specs

Parameters	Direction	Description
dwIndex	in	index into the stream
hClientGroup	out	client group handle
hClient	out	client item handle
wQuality	out	quality
ftTimeStampItem	out	timestamp
vtValue	out	value

HRESULT Return Codes

Table 53 GetByIndex (for OPCSTMFORMATDATETIME) Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful
HCI_E_BADSTREAMFORMAT	Bad stream format
HCI_E_INVALIDINDEX	dwIndex is beyond the range of the number of elements in the stream

6 Custom Interface APIs

6.7 HciStreamUtil Class

GetByIndex (for OPCSTMFORMATDATA)

```
STDMETHODIMP GetByIndex (DWORD          dwIndex,  
                        OPCHANDLE       &hClientGroup,  
                        OPCHANDLE       &hClient,  
                        WORD             &wQuality,  
                        VARIANT &vtValue);
```

Description

GetByIndex is used to retrieve data from the stream by an index into the stream of the format OPCSTMFORMATDATA. Use GetItemCount or GetGroupHeader to determine the number of items returned in the stream.

Parameter Specifications

Table 54 GetByIndex (for OPCSTMFORMATDATA) Parameter Specs

Parameters	Direction	Description
dwIndex	in	index into the stream
hClientGroup	out	client group handle
hClient	out	client item handle
wQuality	out	quality
vtValue	out	value

HRESULT Return Codes

Table 55 GetByIndex (for OPCSTMFORMATDATA) Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful
HCI_E_BADSTREAMFORMAT	Bad stream format
HCI_E_INVALIDINDEX	dwIndex is beyond the range of the number of elements in the stream

GetByIndex (for OPCSTMFORMATWRITECOMPLETE)

```
STDMETHODIMP GetByIndex (DWORD          dwIndex,  
                        OPCHANDLE       &hClientGroup,  
                        OPCHANDLE       &hClient,  
                        HRESULT &dwError);
```

Description

GetByIndex is used to retrieve data from the stream by an index into the stream of the format OPCSTMFORMATWRITECOMPLETE. Use GetItemCount or GetGroupHeader to determine the number of items returned in the stream.

Parameter Specifications

Table 56 GetByIndex (for OPCSTMFORMATWRITECOMPLETE) Param Specs

Parameters	Direction	Description
dwIndex	in	index into the stream
hClientGroup	out	client group handle
hClient	out	client item handle
dwError	out	the HRESULT for the item that was written

HRESULT Return Codes

Table 57 GetByIndex (for OPCSTMFORMATWRITECOMPLETE) Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful
HCI_E_BADSTREAMFORMAT	Bad stream format
HCI_E_INVALIDINDEX	dwIndex is beyond the range of the number of elements in the stream

GetByHandle (for OPCSTRMFORMATDATETIME)

```
STDMETHODIMP GetByHandle (OPCHANDLE   opcHandle,  
                           OPCHANDLE   &hClientGroup  
                           OPCHANDLE   &hClient,  
                           WORD          &wQuality,  
                           FILETIME     &ftTimeStampItem,  
                           VARIANT &vtValue);
```

Description

Used to retrieve data from the stream by client item handle. Stream must be of the format OPCSTRMFORMATDATETIME. Use GetItemCount or GetGroupHeader to determine the number of items returned in the stream.

Parameter Specifications

Table 58 GetByHandle (for OPCSTRMFORMATDATETIME) Param Specs

Parameters	Direction	Description
opcHandle	in	client item handle
hClientGroup	out	client group handle
hClient	out	client item handle
wQuality	out	quality
ftTimeStampItem	out	timestamp
vtValue	out	value

HRESULT Return Codes

Table 59 GetByHandle (for OPCSTRMFORMATDATETIME) Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful
HCI_E_BADSTREAMFORMAT	Bad stream format
HCI_E_INVALIDHANDLE	opcHandle does not exist for the elements contained in this stream

GetByHandle (for OPCSTRMFORMATDATA)

```
STDMETHODIMP GetByHandle (OPCHANDLE  opcHandle,  
                           OPCHANDLE  &hClientGroup,  
                           OPCHANDLE  &hClient,  
                           WORD        &wQuality,  
                           VARIANT     &vtValue);
```

Description

Used to retrieve data from the stream by client item handle. Stream must be of the format OPCSTRMFORMATDATA. Use GetItemCount or GetGroupHeader to determine the number of items returned in the stream.

Parameter Specifications

Table 60 GetByHandle (for OPCSTRMFORMATDATA) Param Specs

Parameters	Direction	Description
opcHandle	in	Client item handle
hClientGroup	out	Client group handle
hClient	out	Client item handle
wQuality	out	Quality
vtValue	out	Value

HRESULT Return Codes

Table 61 GetByHandle (for OPCSTRMFORMATDATA) Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful
HCI_E_BADSTREAMFORMAT	Bad stream format
HCI_E_INVALIDHANDLE	opcHandle does not exist for the elements contained in this stream

GetByHandle (for OPCSTRMFORMATWRITECOMPLETE)

```
STDIMP METHODIMP GetByHandle (OPCHANDLE opCHandle,  
                             OPCHANDLE &hClientGroup,  
                             OPCHANDLE &hClient,  
                             HRESULT dwError);
```

Description

Used to retrieve data from the stream by client item handle. Stream must be of the format OPCSTRMFORMATWRITECOMPLETE. Use GetItemCount or GetGroupHeader to determine the number of items returned in the stream.

Parameter Specifications

Table 62 GetByHandle (OPCSTRMFORMATWRITECOMPLETE) Param Specs

Parameters	Direction	Description
opCHandle	in	Client item handle
hClientGroup	out	Client group handle
hClient	out	Client item handle
dwError	out	HRESULT for the item that was written

HRESULT Return Codes

Table 63 GetByHandle (OPCSTRMFORMATWRITECOMPLETE) Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful
HCI_E_BADSTREAMFORMAT	Bad stream format
HCI_E_INVALIDHANDLE	opCHandle does not exist for the elements contained in this stream

Comments

None

GetGroupHeader

```
STDMETHODIMP GetGroupHeader (DWORD    &dwSize,  
                             DWORD    &dwItemCount,  
                             OPCHANDLE &hClientGroup,  
                             DWORD    &dwTransactionID,  
                             HRESULT &hrStatus);
```

Description

Used to retrieve the group header from the stream.

Parameter Specifications

Table 64 GetGroupHeader Parameter Specifications

Parameters	Direction	Description
dwSize	out	Stream length in bytes
dwItemCount	out	Number of items in stream
dwTransactionID	out	ID of asynch transaction
hrStatus	out	Status of advise

HRESULT Return Codes

Table 65 GetGroupHeader HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful

Comments

None.

6 Custom Interface APIs

6.7 HciStreamUtil Class

GetItemCount

```
STDMETHODIMP GetItemCount (DWORD &dwItemCount);
```

Description

Used to retrieve the count of items in this stream.

Parameter Specifications

Table 66 GetItemCount Parameter Specifications

Parameters	Direction	Description
dwItemCount	out	Number of items in stream

HRESULT Return Codes

Table 67 GetItemCount HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful

Comments

None.

GetStreamFormat

```
const OPCSTREAMFORMAT GetStreamFormat();
```

Description

Used to retrieve the Stream format of this stream.

Comments

None.

GetTransID

```
STDMETHODIMP GetTransID(DWORD &dwTransID);
```

Description

Retrieves the transaction ID located in the Groupheader.

Parameter Specifications

Table 68 GetTransID Parameter Specifications

Parameters	Direction	Description
dwTransID	out	Transaction ID

HRESULT Return Codes

Table 69 GetTransID HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
E_FAIL	The function was not successful

Comments

None.

6 Custom Interface APIs

6.7 HciStreamUtil Class

7. Automation Interface APIs

7.1 Introduction

Overview

This section describes how the custom interface is exposed through the IDispatch interface of OLE Automation. The IDispatch interfaces are fully supported by VC++ and Visual Basic (VB). They allow any application that has an OLE Automation Controller (e.g., VB, VC++, VBA in Excel and MS Project, Visio, etc.) to access the OPC Interface.

HCI wrapper classes

In addition to the OPC objects noted above, HCI provides a wrapper class: CCHciVBHelper. This allows automation clients access to HCI functions.

7.2 ICHciVBHelper Interface

Functional description

The ICHciVBHelper interface provides an easy way for automation clients to access some essential HCI calls and remote machine operations not possible in Visual Basic. The methods that are provided by this class allow automation clients to access HCI servers in remote machines and to get HCI component information.

Development notes

Visual Basic clients can access this class by including the clientutil 1.0 type library into a project, and declaring a variable of type CHciVBHelper, as shown below.

```
Dim m_helper As New CHciVBHelper
```

Clients that cannot access type libraries could use the following commands:

```
Dim m_helper as object  
Set m_helper = CreateObject("CHciVBHelper.CHciVBHelper")
```



ATTENTION

ICHciVBHelper is the COM version of the CCHciVBHelper class which was previously available in hciatl.dll. IHciVBHelper is an interface of HciClientUtil COM object. This COM object is provided within clientutil.dll. The names and functionality of the methods are the same as those provided by CCHciVBHelper class. If your VB client currently includes hciatl type library, you must rebuild the client to include clientutil 1.0 type library instead since the hciatl type library no longer exists.

Public Methods summary

- CreateHciObject
- GetHciComponentInfo
- UTCToLocal
- GetAttributes
- Start
- GetInfo
- CreateRemoteObject
- GetHciStatus
- SetAttributes
- Configure
- CancelPulse
- Checkpoint

CreateHciObject

```
Function CreateHciObject(bstrComponentID As String, [vHostName]) As Object
```

Description

Given the name of an HCI component, return an instance of the server object. The instance is returned as the return value to the client.

Parameter Specifications

Table 70 CreateHciObject Parameter Specifications

Parameters	Direction	Description
bstrComponentID	in	Name of the HCI component
vHostName	in	Optional hostname where the interface is to be requested. If not specified, then the local host is used.
ppDisp	Returned value	Pointer to IDispatch interface

HRESULT Error Codes

Table 71 CreateHciObject HRESULT Error Codes

Error Code	Description
E_FAIL	Unable to create the component
et al	Errors returned from CoCreateInstanceEx

This method encapsulates the IHciClientUtil->GetComponentInfo and CoCreateInstanceEx calls required to create an Hci component and return a IDispatch pointer to a Visual Basic client. Refer to section “Custom Interface APIs” for information regarding IHciClientUtil->GetComponentInfo.

Example

```
'CreateHciObject - Creates an instance of an HCI component
Dim helper As New CHciVBHelper
Dim server As Object
Set server = helper.CreateHciObject("template_server1")
```

7 Automation Interface APIs

7.2 ICHciVBHelper Interface

CreateRemoteObject

```
Function CreateRemoteObject(bstrComputerName As String,  
bstrProgID As String) As Object
```

Description

Since Visual Basic 5.0 has no built-in call equivalent to CoCreateInstanceEx, CreateRemoteObject is provided to allow an automation object to create and reference objects on remote TPS nodes. However, with VB 6.0, CreateObject lets you specify a machine name as an optional argument, which allows you to create objects on remote machines.

Parameter Specifications

Table 72 CreateRemoteObject Parameter Specifications

Parameters	Direction	Description
bstrComputerName	in	Name of the remote node
bstrProgID	in	ProgID of the component
ppDisp	Returned value	Pointer to IDispatch interface

HRESULT Error Codes

Table 73 CreateRemoteObject HRESULT Error Codes

Error Codes	Description
E_FAIL	Unable to create the remote object
et al	Errors returned from CoCreateInstanceEx

Example

```
'CreateRemoteObject - Create a COM object on a remote computer  
' In VB 6.0 could also use new CreateObject ' command  
Dim helper As New CHciVBHelper  
Dim Server As object  
Set server = helper.CreateRemoteObject("A_Computer", "Hci.DssTemplate")
```

GetComponentInfo

```
Function GetComponentInfo(bstrComponentName As String,  
pbstrCLSID As String, pbstrProgID As String, pbstrHostName As  
String) As Long
```

Description

This method provides a wrapper to the IHciClientUtil->GetComponentInfo call, which provides information about a given Hci component.

Parameter Specifications

Table 74 GetComponentInfo Parameter Specifications

Parameters	Direction	Description
bstrComponentName	in	The name of the component
pbstrCLSID	out	The CLSID of the component
pbstrProgID	out	The ProgID of the component
pbstrHostName	out	The name of the host
pbIsLocal	Returned Value	Boolean value, which is set to TRUE if the component is on the local node

HRESULT Error Codes

Table 75 GetComponentInfo HRESULT Error Codes

Error Code	Description
E_FAIL	The function was unsuccessful

Comments



REFERENCE - INTERNAL

For details regarding the IhciClientUtil->GetComponentInfo, refer to the section "Custom Interface APIs."

7 Automation Interface APIs

7.2 ICHciVBHelper Interface

Example

```
'GetComponentInfo - Get Server component information
Dim bLocalNode As Long
Dim helper As New CHciVBHelper
Dim CLSID As String
Dim ProgID As String
Dim HostName As String
bLocalNode = helper.GetComponentInfo ("template_server1", CLSID,
ProgID, HostName)
```

UTCToLocal

```
Function UTCToLocal(dUTC As Date) As Date
```

Description

Converts UTC to local time.

Parameter Specifications

Table 76 UTCToLocal Parameter Specifications

Parameters	Direction	Description
dUTC	in	A time in UTC format
LocalTime	Returned value	The local time equivalent

HRESULT Error Codes

Table 77 UTCToLocal HRESULT Error Codes

Error Codes	Description
E_FAIL	Unable to convert the time

Example

```
'UTCToLocal - Convert UTC to local time
Dim helper As New CHciVBHelper
MsgBox helper.UTCToLocal(Now())
```

Checkpoint

```
Sub Checkpoint(bstrComponentName As String, [vFileName], [vHostName])
```

Description

Tells a component to checkpoint.

Parameter Specifications

Table 78 CheckPoint Parameter Specifications

Parameters	Direction	Description
BstrComponentName	in	The component name
VFileName	In, Optional	The optional name of the file to checkpoint to
VHostname	In, Optional	The optional name of the hostnode of the component

HRESULT Error Codes

Table 79 CheckPoint HRESULT Error Codes

Error Codes	Description
E_FAIL	Unable to checkpoint

Example

```
'Checkpoint - Tell the component to checkpoint  
Dim helper As New CHciVBHelper  
helper.Checkpoint "template_server1"
```

7 Automation Interface APIs

7.2 ICHciVBHelper Interface

SetAttributes

```
Sub SetAttributes(Server As Object, Names() As String, Values()  
    As Variant, Errors() As Long)
```

Description

This method allows the attributes of an HCI server to be set.

Parameter Specifications

Table 80 SetAttributes Parameter Specifications

Parameters	Direction	Description
Server	in	Pointer to IDispatch interface for the component
Names	in	The list of server attribute names to be set
Values	in	The list of server attribute values to be set
Errors	out	Array of HRESULTs indicating the success of the individual server attribute sets. The errors correspond to the names passed in saAttrNames. This indicates whether the HCI Server accepted the new value of the server attribute. NOTE that any FAILED error code indicates that the HCI server rejected the value.

HRESULT Return Codes

Table 81 SetAttributes HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
S_FALSE	The function was partially successful. See ppErrors to determine server attribute specific errors.
E_INVALIDARG	An invalid argument is being used
E_FAIL	The function was unsuccessful

ppErrors Return Codes

Table 82 SetAttributes ppErrors Return Codes

ppErrors Code	Description
S_OK	The attribute was successfully set
E_ACCESSDENIED	The specific HCI Server has determined that the caller does not have sufficient access rights to set the specified server attribute.
E_NOTIMPL	The specific HCI Server does not support the specified server attribute.
E_FAIL	The specific HCI Server was unable to set the specified server attribute.

Comment

This method provides a wrapper to the IHciAttributes::SetAttributes method. Refer to the section entitled “IHciAttributes Interface” for details. Note that the only way to verify the success of the call is to check the individual error codes returned in the Errors array.

Example

```
Sub SetAttributes()  
    Dim helper as New CHciVBHelper  
    Dim server as object  
  
    Set server = CreateObject("Hci.DssTemplate")  
  
    Dim names(0) As String  
    Dim values(0) As Variant  
    Dim errors() As Long  
  
    names(0) = "Priority"  
    values(0) = 10  
  
    helper.SetAttributes server, names(), values(), errors()  
    if errors(0) = 0 then  
        MsgBox "The priority has been set to " & values(0)  
    Else  
        MsgBox "An Error has occurred"  
    End if  
End Sub
```

7 Automation Interface APIs

7.2 ICHciVBHelper Interface

GetAttributes

```
Sub GetAttributes(Server As Object, Names() As String, Values()  
As Variant, Errors() As Long)
```

Description

GetAttributes retrieves the current value associated with the server attributes specified in saAttrNames.

Parameter Specifications

Table 83 GetAttributes Parameter Specifications

Parameters	Direction	Description
Server	in	Pointer to IDispatch interface for the component
Names	in	The list of server attribute names to be retrieved
Values	out	The list of the current values of server attributes specified by Names
Errors	out	Array of HRESULTs indicating the success of the individual server attribute gets. The errors correspond to the names passed in Names.

HRESULT Error Codes

Table 84 GetAttributes HRESULT Error Codes

Error Code	Description
E_INVALIDARG	An invalid argument is being used
E_FAIL	The function was unsuccessful

ppErrors Return Codes

Table 85 GetAttributes ppErrors Return Codes

ppErrors Code	Description
S_OK	The attribute was successfully retrieved.
E_ACCESSDENIED	The specific HCI Server has determined that the caller does not have sufficient access rights to get the specified server attribute.
E_NOTIMPL	The specific HCI Server does not support the specified server attribute.
E_FAIL	The specific HCI Server was unable to get the specified server attribute.

Comment



REFERENCE - INTERNAL

This method provides a wrapper to the IHciAttributes::GetAttributes method. For details, refer to the section entitled "GetAttributes".

Example

```
Sub GetAttributes()  
    Dim helper as object  
    Dim server as object  
  
    Set helper = CreateObject("CHciVBHelper.CHciVBHelper")  
    Set server = CreateObject("Hci.DssTemplate")  
  
    Dim names(0) As String  
    Dim values() As Variant  
    Dim errors() As Long  
  
    names(0) = "Priority"  
  
    helper.GetAttributes server, names(), values(), errors()  
    MsgBox "The priority is " & values(0)  
End Sub
```

7 Automation Interface APIs

7.2 ICHciVBHelper Interface

Configure

```
Sub Configure(Group As Object, lCycles As Long, lConnection As Long, bStart As Boolean)
```

Description

This method is called to establish and optionally start a single Hci pulse. Once configured and started, the pulse should be sent from the server every period, which is equal to the number of cycles times the update rate of the group. One cycle is equal to the update rate of the group. The client would not usually want to flood the network with pulses at the same rate for which the OnDataChanges occur. If the update rate is high, such as greater than 1 hour, then it may be feasible to have the pulse cycle equal to 1.

In general, if the update rate of an active group is several seconds, then the number of cycles should be 5 or greater. This will keep a tab on the OnDataChange “channel” and alleviate network traffic. A group with a 4 second update rate and a pulse configured for 5 cycles would translate to a pulse from the server to the client at a period of approximately every 20 seconds. If the client misses 3 or more successive periods, then the client may want to setup a new callback reference.

The same connection obtained from the AddCallbackReference on a group must be used as the connection when configuring the pulse. The client can choose to set the pulse to start as part of the configuration. This will save one round trip to the server assuming that the client wants the pulse started after it is configured.

Parameter Specifications

Table 86 Configure Parameter Specifications

Parameters	Direction	Description
Group	in	IDispatch pointer to the group.
lCycles	in	The number of cycles between pulses sent from the server to the client. The length of a cycle is equal to the update rate of the group.
lConnection	in	Connection key established with the AddCallbackReference method.
bStart	in	If bStart is TRUE, the Pulse will automatically be started. Otherwise if the value is FALSE, then the Pulse will not be started until Start is called. FALSE will not cause a CancelPulse if currently running.

HRESULT Error Codes

Table 87 Configure HRESULT Error Codes

Error Code	Description
E_FAIL	The function was unsuccessful. Callback has not been setup via DAdvise.
E_NOINTERFACE	The function was unsuccessful. The IHciPulse interface does not exist on the server.
E_INVALIDARG	The function was unsuccessful. The number of cycles is equal to zero, or the dispatch pointer is invalid.

Comment



REFERENCE - INTERNAL

For an overview of the IHciPulse interface, refer to the section titled "IHciPulse Interface."

Example

```
'Configure - Sets up HCI pulse for OPC 1.0a callback
'
  Dim connection As Long
  Dim Cycles As Long
  Dim start As Boolean
  Cycles = 4 'callback will fire 4 cycles regardless of datachange
  start = False
  helper.Configure g, Cycles, connection, start
```

7 Automation Interface APIs

7.2 ICHciVBHelper Interface

Start

```
Sub Start(Group As Object, lConnection As Long)
```

Description

This method is called to start a configured pulse. The connection is used to determine which pulse to start. The connection is obtained when the client calls the AddCallbackReference method.

Parameter Specifications

Table 88 Start Parameter Specifications

Parameters	Direction	Description
Group	in	IDispatch pointer to the group
lConnection	in	Connection key established with the AddCallbackReference method

HRESULT Error Codes

Table 89 Start HRESULT Error Codes

Error Code	Description
E_INVALIDARG	The function was unsuccessful. Invalid dispatch pointer.
E_NOINTERFACE	The function was unsuccessful. The IHciPulse interface does not exist on the server.
E_FAIL	The function was unsuccessful. The connection does not exist.

Comments



REFERENCE - INTERNAL

For an overview of the IHciPulse interface, refer to section "IHciPulse Interface."

Example

```
'Configure pulse (see above)
'Start - Start the pulse
helper.start g, connection
```

CancelPulse

Sub CancelPulse(Group As Object, lConnection As Long)

Description

This method is called to explicitly stop a particular Hci pulse. If a group is set from active to inactive, then all pulses on that group will be stopped, just as if CancelPulse was called for each pulse on that group.

Parameter Specifications

Table 90 CancelPulse Parameter Specifications

Parameters	Direction	Description
Group	in	IDispatch pointer to the group
lConnection	in	Connection key established with the AddCallbackReference method

HRESULT Return Codes

Table 91 CancelPulse HRESULT Return Codes

Return Code	Description
S_OK	The function was successful
E_INVALIDARG	The function was unsuccessful. Invalid dispatch pointer.
E_NOINTERFACE	The function was unsuccessful. The IHciPulse interface does not exist on the server.
E_FAIL	The function was unsuccessful. The connection does not exist.

Comments



REFERENCE - INTERNAL

For an overview of the IHciPulse interface, refer to the section "IHciPulse Interface."

7 Automation Interface APIs

7.2 ICHciVBHelper Interface

Example

```
'Configure pulse (see above)
'CancelPulse - Cancels the specified pulse
helper.CancelPulse g, connection
```

GetInfo

```
Sub GetInfo(Group As Object, bActive As Boolean, lUpdateRate As Long,
lCycles As Long)
```

Description

This method is called to retrieve the current configuration of the pulse associated with the group. The information will include the activeness of the group, the group update rate and number of cycles between pulses.

Parameter Specifications

Table 92 GetInfo Parameter Specifications

Parameters	Direction	Description
Group	in	IDispatch pointer to the group
bActive	out	If the group is active, then TRUE is returned. Otherwise FALSE is returned.
lUpdateRate	out	The current update rate of the group in milliseconds
lCycles	out	The number of cycles between pulses. One cycle is equivalent to the update rate.

HRESULT Return Codes

Table 93 GetInfo HRESULT Return Codes

Return Code	Description
E_FAIL	The function was unsuccessful. The connection does not exist.
E_POINTER	The function was unsuccessful. An invalid pointer was passed to the method.
E_INVALIDARG	The function was unsuccessful. It could be that an invalid dispatch pointer was passed to the method or the number of configured pulses is zero.
E_NOINTERFACE	The function was unsuccessful. The IHciPulse interface does not exist on the server.
E_OUTOFMEMORY	The function was unsuccessful. The server process ran out of memory.

7 Automation Interface APIs

7.2 ICHciVBHelper Interface

Comments



REFERENCE - INTERNAL

For an overview of the IHciPulse interface, refer to the section "IHciPulse Interface."

Example

```
'Configure pulse (see above)
'GetInfo - Get Pulse information for a group
  Dim bActive As Boolean
  Dim UpdateRate As Long
  Dim Cycles As Long

  helper.GetInfo g, bActive, UpdateRate, Cycles
```

7.3 Creating an OPC Server Object

Description

An object that has a connection to a running OPC Server, such as a SystemOPCServer. You must create an OPCServer object before you can create other objects such as groups and items.

The OPCServer supports several different Dispatch Interfaces.

Example:

```
Dim MyServer as Object
Dim OPCServer as IOPCServerDisp

'If the server needs to be created on a remote node, use the 'VB
helper as shown in the following lines of code

'Create the VB Helper
Dim HCI_VBhelper As Object
Set HCI_VBhelper = CreateObject("CHciVBHelper.CHciVBHelper.1")

'Create a remote OPCServer
Dim MyServer as Object
Set MyServer =
HCI_VBhelper.CreateRemoteObject("AnyHost", "VendorX_OPCTServer.1")

'Otherwise use the CreateObject command
'Set MyServer = CreateObject("VendorX_OPCTServer.1")

Set MyServer = CreateRemoteObject("AnyHost", "VendorX_OPCTServer.1")
Set OPCServer = MyServer
```

7 Automation Interface APIs

7.3 Creating an OPC Server Object

8. HCI Client Development Guidelines

8.1 Debugging Your HCI Clients

How to debug

The client's executables found under `\ProgramFiles\Honeywell\Tps\Hci\` are all built using the release compilation flags. The source code for all the sample clients is provided under the HCI Client Development toolkit `\ProgramFiles\Honeywell\Tps\Toolkits\Hci\SampleClients`. To debug a client, you need to build a debug version of it. Within the Visual Studio environment, load the desired client's project, build its WinDebugIU target, and step through the code. You can run a debug version of a client against a release version of the server and vice-versa.

8.2 Quickstart Client – Server Communication

Permissions and bypassing them

Configuring and managing the HCI Servers by the TPS system requires complete system installation and configuration. Additionally, when a managed server is launched, it contacts the Component Admin Service (CAS) to request permission to run. To bypass this mechanism and to de-couple the server from System Management, you can add a "TPSManaged" key value name with a value of "BypassSSM" under the target server configuration key found under:

`HKEY_LOCAL_MACHINE\Software\Honeywell\HCIComponents\HCIComponentName`. Note that for local components, the HCI Name service overwrites this area of the registry each time it replicates the component configuration. This causes the BypassSSM value to be overwritten.

8.0 HCI Client Development Guidelines

9. OPC Performance Guidelines

9.1 Introduction

Overview

The OLE for Process Control interfaces is a means for developers to access process data through a standard interface. Various different vendors, allowing access to a variety of data sources, will implement these interfaces. Honeywell's OPC servers will share one common component, namely a Generic OPC dynamic link library. The OPC servers will be accessible through the OPC interfaces, which use COM. The rules for writing fast COM clients apply to OPC clients as well. In addition, there are some techniques of using the OPC interfaces that are faster than others use.

What follows are some guidelines for writing fast OPC clients. The data used was generated using the Honeywell Template server that is provided in the HCI toolkit.

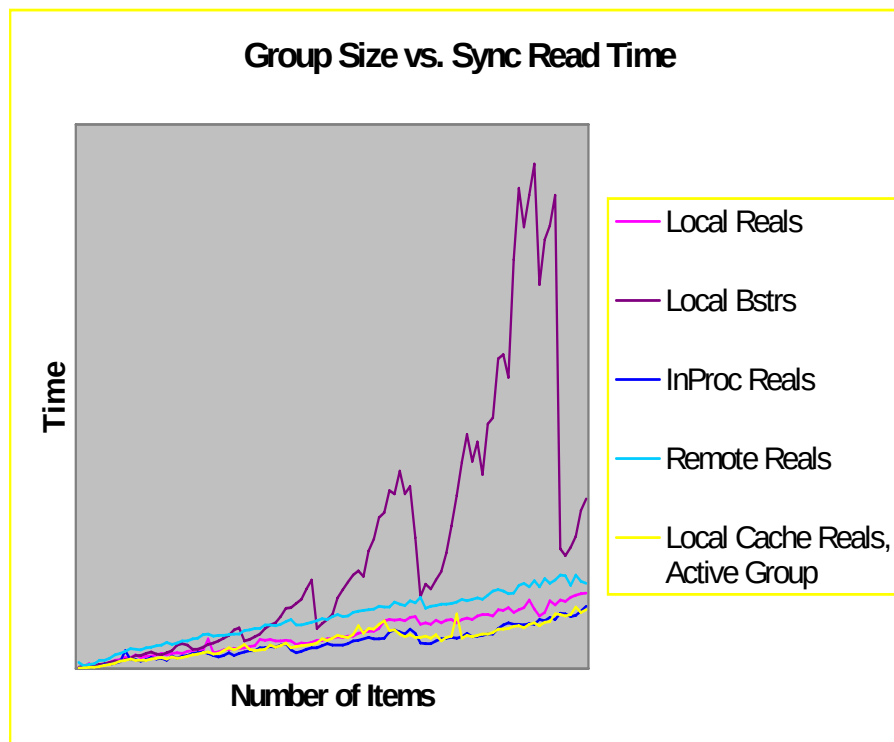
9.2 Achieving Fast Performance with COM

Comparing In-Process to Local Reals

All OPC client developers use COM. The following chart confirms some of the basic COM rules. (Timing ten synchronous device reads generated the chart.)

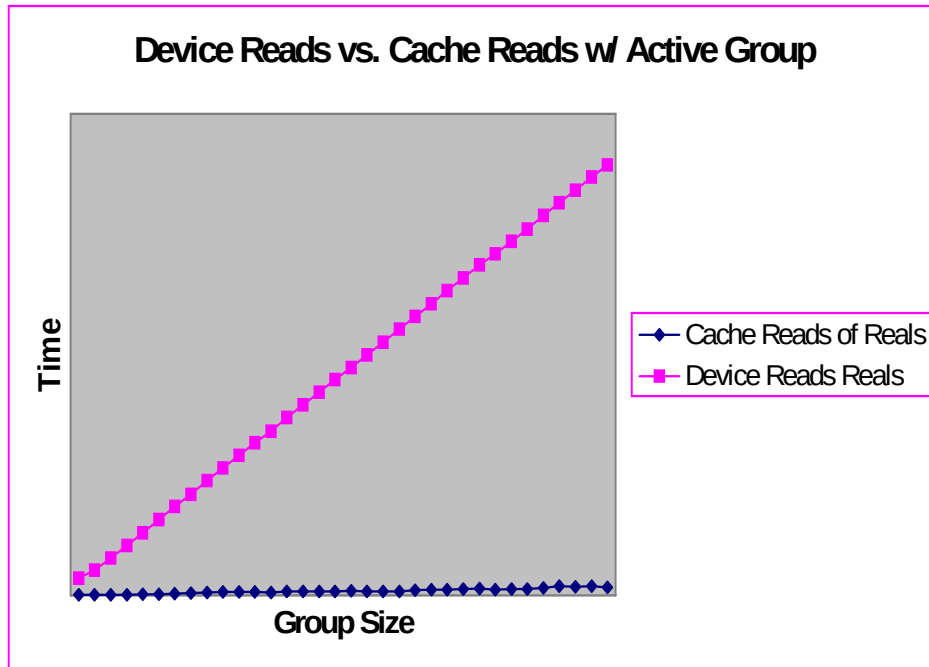
Note that in-process (InProc Reals) is faster than local is (Local Reals), and both are faster than remote calls (Remote Reals). Also note that reading "reals" is faster than reading "strings." The last thing to note is the "Local Cache Reals, Active Group" line, which gives us the first OPC optimization. All other reads were from the device; however, reading from the cache with an active group was faster than local device reads. The active group was updating the cache, so the cached data was always fresh. The performance difference may not seem very significant in this chart due to the usage of the HCI sample server, DSS Template, which reads its data from memory.

Figure 5 Comparing Group Size vs. Sync Read Time



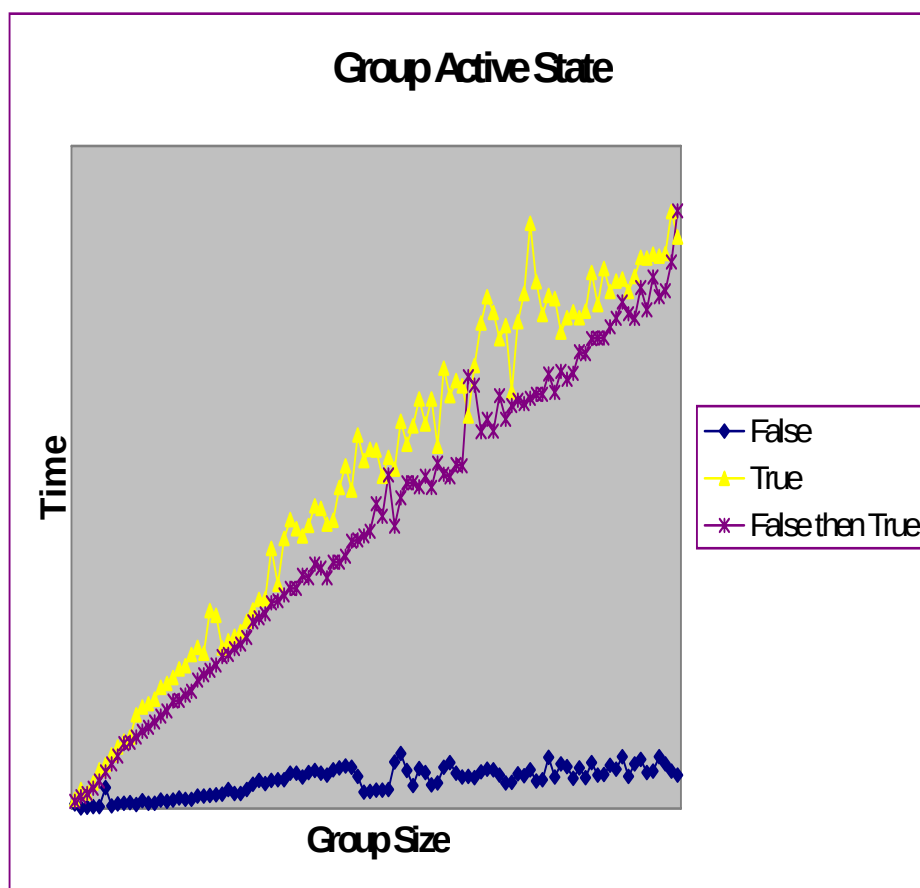
The following is the same comparison of reading from the device and reading from cache with an active group. This time the DSS Template made device access slightly more expensive by sleeping for 1 millisecond on each device read. Clearly, it is faster to read items from cache with an active group.

Figure 6 Comparing Device Reads vs. Cache Reads



The following chart shows that it is faster to add items to an inactive group and activate the group after all points have been added, than it is to add items to an active group.

Figure 7 Viewing Group Active State



9.3 Summary

COM rules to follow for optimal performance

In conclusion, the basic COM rules apply to any OPC client:

- In-process is faster than out-of-process (local), which is faster than remote.
- OPC servers provide a means for clients to optimize performance by using cache reads from an active group versus device reads.
- When setting up an active group, it is more efficient to add the active items to the group while it is inactive, and then activate the group after all items have been added than to add active items to the group while it is active.
- The data also indicates that reading "reals" is faster than reading "strings" (enumerations).

9.4 Troubleshooting

Possible symptoms and solutions

The following table lists possible HCI Client or Server failure symptoms followed by various solutions and symptoms you should check.

Symptom	Solution
TPS Status Display fails to show the view of the TPSDomain	1. Verify that the TPSAdmin is running by using the NT Task Manager. 2. Check the event log for possible failure reasons. For example, if the Event Log states that TPSRepl\$ is not accessible, from a command window type "net share" and ensure that TPSRepl\$ is listed. If listed, try "dir //ServerHostName/TPSRepl\$" and ensure that the directory is accessible. 3. Check the TPS Domain configuration to which the current node belongs. You can do so by reviewing the TPS Configuration procedure. 4. Using the TPS configuration Utility, make sure that the replication operation succeeded. If you are not sure, perform a replication again then restart the TPSAdmin.

Symptom	Solution
HCI Server fails to start.	1. Check the Event Log (both System and Application) for possible failure reasons. 2. Check the server's configuration in the registry under HKEY_LOCAL_MACHINE\Software\Honeywell\MyTpsDomain\HciComponents\ComponentName . Does the component have a valid ProgId? 3. Check the registry for improper self-registration of the server and its associated proxy/stubs. All HCI components have a ProgId that starts with Hci. 4. If the component is local, temporarily change its Run As identity (using dcomcnfg) to Interactive. This will simplify the DCOM security checking. 5. Check the server's security settings in dcomcnfg. Temporarily add System, Interactive, Your User account, and/or Everyone to the list. 6. Avoid mixing debug and non-debug versions of the component and the HCI Runtime. Upon installation, the non-debug version of the HCI runtime is registered 7. If attempting to startup a remote HCI server fails, then check the security settings on DCOM and the component. Try running both the client and server on the same machine. First, run both on the server machine to make sure they work fine locally. Then follow the same on the client machine. Try logging in to both machines as the same user and temporarily set the component Run As property to Interactive.
Server starts, but client does not execute with success. If using sample HCI clients and tests fail.	1. Verify that you are using the latest O/S upgrades. 2. A client with a callback is actually two separate COM connections between the server and the client. If callbacks fail, then ensure that both the client account and the Server Run As account are in the Default Security tab within dcomcnfg, and ensure that the server is setup to use the default security permissions. 3. Ensure that all Proxy/Stub DLLs required by HCI Runtime and by your HCI Component are correctly registered. All self-registered HCI components and Proxy/Stubs could be found in one directory: \ProgramFiles\Honeywell\Tps\Hci\ 4. The HCIRuntime and HCI Components are currently built and run against the same HCI release number. To check a release number of a specific component or DLL, look at the version info in the component's property using the Windows Explorer.

Honeywell

Honeywell International
Process Solutions
2500 West Union Hills Drive
Phoenix, AZ 85027